

ANDREAS SCHÄFER

THEORY OF COMPUTING

TH LÜBECK

1

Regular Languages and Finite Automata

A SIMPLE TRAFFIC LIGHT AT A CROSSING SHOWS A GREEN LIGHT EITHER FOR THE CAR TRAFFIC OR FOR THE PEDESTRIANS. It has two states and switches between them. More complex traffic lights can be realized using the same idea with more states and transitions. This concept of a control with states and transitions is omnipresent in vending machines, CPUs to the TCP protocol. *Finite automata* are a simple model, can be represented graphically for small systems and can be easily implemented in hardware and software. Finite automata are also used in the background for analyzing user input in web forms as well as in compilers.

1.1 Alphabet, Word, Language

Everybody has probably an intuitive understanding of the notions alphabet, word, language. In computer science these notions are particularly important. Programs process input from users or other programs. The program code itself is a word in a programming language. Therefore, we take a closer look.

Definition 1: Alphabet

An **alphabet** Σ is a finite non-empty set. The elements of an alphabet are called letters.

Definition 2: Word

A **word** $w = a_1 \dots a_n$ is a sequence of letters from an alphabet.

Definition 3: Language

A **language** L is finite or infinite set of words.

In Java strings can be concatenated with the $+$ -Operator. Concatenating the empty string $""$ does not alter the string.

Definition 4: Operations on words

- ε or the **empty word**
- $w_1 \cdot w_2$ or short $w_1 w_2$ denotes the **concatenation** of w_1 and w_2

Welcome (Willkommen)



<https://youtu.be/ZK7DWDGavL4>
(<https://youtu.be/QIzbaRnjehA>)



Figure 1.1: Simple traffic light control

Alphabet, Word, Language (Alphabet, Wort, Sprache)



<https://youtu.be/A5BZQkNZ7zs>
(<https://youtu.be/y0xop6RaHvs>)

- w^n denoted the word $\underbrace{w \dots w}_{n\text{-times}}$ with $w^0 = \varepsilon$.
- $|w|$ denotes the **length** of w
- Σ^* is the set of words over the alphabet Σ (incl. ε).

The concatenation can be lifted to languages and iterated.

Definition 5: Concatenation of Languages

- $L_1 \circ L_2 = \{w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$ (Concatenation of languages)
- $L^0 = \{\varepsilon\}$
- $L^n = \underbrace{L \circ \dots \circ L}_{n\text{-mal}} = \{w_1 \dots w_n \mid w_i \in L, 1 \leq i \leq n\}$
Alternative Definition: $L^{n+1} = L^n \circ L$
Set of words that are obtained by concatenating n words from L

In the languages L^n are all the words that are obtained by concatenating n words from L . In many situations we need the set of words that are obtained by concatenating an arbitrary number of words from L .

Definition 6: Star

Let L be a language. The language of words obtained by concatenating an arbitrary (incl. 0) number of words from L is defined by

$$L^* = \bigcup_{i=0}^{\infty} L^i.$$

Note: The empty word ε is always in the L^* by definition.

If we want to rule out zero concatenations, we can use the plus operator instead of the star.

Definition 7: Plus

Let L be a language. The language of words obtained by concatenating more than zero words from L is defined by

$$L^+ = \bigcup_{i=1}^{\infty} L^i.$$

The empty word is only in L^+ if it is already in L .

Operations on Words (Operationen auf Wörtern)



<https://youtu.be/QIzbaRnjehA>
(<https://youtu.be/PNpZfJYfaHk>)

Operation on Languages (Operationen auf Sprachen)



<https://youtu.be/PBkeisxeJ3s>
(https://youtu.be/CkDP_zqekgQ)

1.2 Deterministic Finite Automata

Finite automata are used to analyze user input from simple web forms to compilers for programming languages. The automata read the input letter by letter and use the states to “memorize” the knowledge about the string that it read so far.

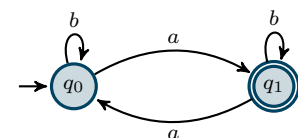


Figure 1.2: DFA A accepting $L(A) = \{w \in \{a, b\}^* \mid \#_a(w) \text{ is odd}\}$

The automaton in 1.2 has two states. It is in q_0 if the number of a read so far is even and in q_1 if it is odd. Reading an a leads to change of state whereas reading a b does not. It accepts all words that lead to an accepting state (double circle) if all letters of the word are processed.

The graphical representation is easy to grasp, for constructions an algorithms working on automata, we use a formal definition as a structure. Such structures are also usually found in programming languages like or even in Java in which they evolved to classes.

Definition 8: Deterministic Finite Automaton (DFA)

A **deterministic finite automaton (DFA)** A is a structure $A = (Q, \Sigma, \delta, q_0, F)$ where

- Q is a finite set of **states**,
- Σ is the **input alphabet**
- $\delta: Q \times \Sigma \rightarrow Q$ is the **transition function**
- $q_0 \in Q$ is the **initial state**,
- $F \subseteq Q$ is the set of **accepting (or final) states**.

Finite automata read the input letter by letter and taking transitions and changing state accordingly. If the last letter is read and the automaton is in an accepting state, the input word is accepted. The sequence of states that the automaton is in while reading the input is called run.

Definition 9: Run (DFA)

Let $w = a_1 a_2 \dots a_n$ be a word and $A = (Q, \Sigma, \delta, q_0, F)$ a DFA. A **run** of A for w is a sequence of states $q_0, q_1, \dots, q_n$ such that

- q_0 is the initial state of A and
- $\delta(q_{i-1}, a_i) = q_i$, i.e. from state q_{i-1} there is transition with letter a_i to state q_i .

We write $q_0 \xrightarrow{w} q_n$ if a run starting in q_0 and ending in q_n exists.

A run is **accepting** if and only if $q_n \in F$, i.e. the last state is an accepting state.

Note: For a DFA there is always exactly one successor state for each letter, therefore there is exactly one run for each word.

Definition 10: Acceptance

Let $w = a_1 a_2 \dots a_n$ be a word and $A = (Q, \Sigma, \delta, q_0, F)$ a DFA. The DFA A accepts the word w if and only if there is an accepting run for w . The **accepted language** is the set of all accepted words

$$L(A) = \{w \in \Sigma^* \mid q_0 \xrightarrow{w} q_f \text{ and } q_f \in F\}.$$

Finite Automata (Endliche Automaten)



<https://youtu.be/9hsXXGabki8>
(https://youtu.be/FGccyPXY_x4)

Finite Automata – Formal Definition (Endliche Automaten – Formale Definition)



<https://youtu.be/5FMku9912p4>
(<https://youtu.be/bJtM3Ze1yvc>)

How DFAs work (Wie DFAs arbeiten)



<https://youtu.be/gqmxCPKF-VE>
(<https://youtu.be/3iS5wB9toFc>)

Definition 11: Regular Language

A formal language is called *regular* if and only if there is a deterministic finite automaton that accepts the language.

Constructing DFAs

To devise a DFA for a given language, you should determine what you have to memorize while reading the letters of the input.

Check for Subword Memorize how much of the subword has already been read.

Check for a pattern Memorize in which part the pattern the input is.

Counting Letters Memorize the number of letters up to some boundary or modulo a fixed number.

In many cases you can proceed from the initial state letter by letter and consider what the automaton knows about the input so far.

1.3 Nondeterministic finite Automata

The behavior of deterministic finite automata (DFA) is uniquely determined. In each state there is exactly one possible successor state for each possible input letter. Therefore, there is exactly one run for every word.

However, giving automata a choice leads to *nondeterministic finite automata* (NFAs). This ability to choose can greatly simplify an automaton, often requiring only a small number of states. Being concise and easy to grasp makes this model well-suited for the specification. However, in contrast to deterministic finite automata, they cannot be implemented in hardware or software directly.

The formal definition of NFAs is very similar to the definition of DFAs. The differences are that

- NFAs have a (non empty) set of initial states instead of one
- the transition function for NFAs yields a set of successor states instead of exactly one - the set can be empty.

Definition 12: Nondeterministic Finite Automaton (NFA)

A **nondeterministic finite automaton (NFA)** A is a structure $A = (Q, \Sigma, \delta, Q_0, F)$ where

- Q is a finite set of states,
- Σ is the input alphabet
- $\delta: Q \times \Sigma \rightarrow \mathcal{P}(Q)$ is the transition function
- $Q_0 \subseteq Q$ is the **non-empty set of initial states**,

First Letter = Last Letter (*Erster Buchstaben = Letzter Buchstabe*)



<https://youtu.be/kbadZjUug6k>
(<https://youtu.be/kXX-cyup7Wg>)

DFAs count (*DEAs zählen*)



https://youtu.be/MLiFRW_sANE
(<https://youtu.be/DFE0kR5ybbk>)

Introducing Guessing: Nondeterministic Finite Automata (NEAs: Automaten, die raten)



https://youtu.be/MLiFRW_sANE
(<https://youtu.be/Bv-QHc6KI0U>)

- $F \subseteq Q$ is the set of accepting (or final) states.

Due to the nondeterministic choice, an NFA can have multiple runs for an input word. An NFA can also get stuck and therefore may also have no run for an input word.

It accepts an input word, if there is a run that ends in an accepting state.

Definition 13: Run (NFA)

Let $w = a_1 a_2 \dots a_n$ be a word and $A = (Q, \Sigma, \delta, Q_0, F)$ an NFA. A **run** of A for w is a sequence of states $q_0, q_1, \dots, q_n$ such that

- $q_0 \in Q_0$ is an initial state of A and
- $\delta(q_{i-1}, a_i) \ni q_i$.

We write $q_0 \xrightarrow{w} q_n$ if a run starting in q_0 and ending in q_n exists.

A run is accepting if and only if $q_n \in F$, i.e. the last state is an accepting state.

Note: For an NFA there might be no run or more than one run for a given input.

Due to the ability to choose a transition among multiple possibilities, NFAs are a generalization to DFAs. Surprisingly, both have equivalent expressive power. For each NFA we can construct a DFA that accepts the same language. The key idea for this construction is that the DFA memorizes in which states the NFA could be. Each state of the DFA represents the set of states the NFA could be in. The set of states of the DFA is therefore the power set of the set of states of the NFA, which is still finite.

Theorem 14: Rabin and Scott – Subset Construction

Let $A_N = (Q_N, \Sigma, \delta_N, Q_{0N}, F_N)$ be a nondeterministic finite automaton. Construct a deterministic finite automaton $A_D = (\mathcal{P}(Q), \Sigma, \delta_D, Q_{0D}, F_D)$ where

- $\mathcal{P}(Q_N) = \{S \mid S \subseteq Q_N\}$, i.e. the states of the DFA A_D correspond to sets of states of the NFA A_N ,
- $\delta_D(\{q_1, \dots, q_k\}, a) = \bigcup_{i=1}^k \delta_N(q_i, a)$, i.e., the deterministic successor in A_D of the state $\{q_1, \dots, q_k\}$ with letter a is the set of all states of A_N that can be reached with a from a state in $\{q_1, \dots, q_k\}$ in A_N ,
- $F_D = \{S \subseteq Q \mid S \cap F_N \neq \emptyset\}$, i.e., A_D accepts if the state contains an accepting state of A_N .

Then A_D and A_N accept the same language.

The drawback of this construction is the exponential blowup of the state space. However, in many practical cases only a small fraction is actually reachable from the initial state. To avoid the construction of unreachable states, we use algorithm ??.

NFAs: Formal Definition (NEAs: Formale Definition)



<https://youtu.be/U--WfHVooYE>
(<https://youtu.be/9LQFDv22akY>)

From NFA to DFA - Subset Construction (Vom NEA zum DEA Potenzmengenkonstruktion)



<https://youtu.be/zombtiFPXac>
(<https://youtu.be/1ZhMqCQ9UUG>)

1.4 Closure Properties

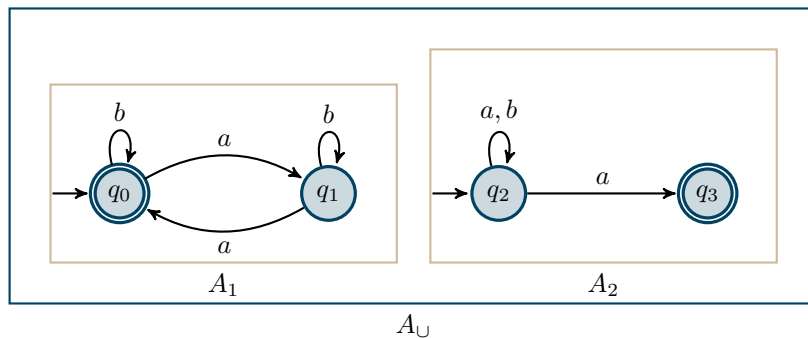
There are a lot operations on languages, from simple set theoretic union and intersection to more language oriented operations like concatenation and star.

It turns out that for each of these operations, there is a corresponding systematic construction for finite automata. Formally, this means that the languages accepted by finite automata are closed under these operations.

Adding two natural numbers yields a natural number. Similarly, the union of two regular languages is a regular language. This property is called a closure property. The regular languages are called “closed under union”.

Closure under Union

A password is usually considered safe if it contains many different types of letters, like uppercase, lowercase, numbers, special characters. Alternatively, using a long word makes a safe password too, even if the word contains only one type of symbols. The language of safe password is therefore the union of the languages for both alternatives.



Closure Properties (Abschlussseigenschaften)



<https://youtu.be/WCC9WrHS99s>
(<https://youtu.be/pwbKUDD0oZE>)

Figure 1.3: Closure under Union

We assume that we have an automaton checking that a word contains different types of symbols and an automaton checking that a word is long. Then, we can simply combine both automata and obtain a nondeterministic automaton that chooses initially to start the first or second automaton.

The informal idea as depicted in figure 1.3 is to simply combine the automata. Non-deterministic choice selects which automaton to use to recognize a word. The precise definition of the construction is given by the following theorem.

Theorem 15: Closure under Union

Let $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ and $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ be non-deterministic finite automata. Define $A_U = (Q_U, \Sigma, \delta_U, Q_{0U}, F_U)$ by

- $Q_U = Q_1 \cup Q_2$ (disjoint union)
- $F_U = F_1 \cup F_2$

Closure under Union (Abschluss unter Vereinigung)



<https://youtu.be/0iICYHMQrd0>
(https://youtu.be/7Qg_jreqi-s)

Closure under Union – Formal Definition (Abschluss unter Vereinigung - Formale Definition)



<https://youtu.be/He9V7GNe2A8>
(<https://youtu.be/XLwHrF1FIgw>)

- $Q_{0\cup} = Q_{01} \cup Q_{02}$
- $\delta_{\cup} = \delta_1 \cup \delta_2$.

Then $L(A_{\cup}) = L(A_1) \cup L(A_2)$.

Closure under Complement

In many cases, user input may not contain special characters or sequences. The language is the complement of the language words that do contain these characters or sequences.

To complement a finite automaton, we have to make it deterministic first. Then each word has exactly one run either ending in an accepting or a non accepting state. Switching accepting and non accepting states complements the accepted language. This idea is illustrated in Figure 1.4.

Theorem 16: Closure under Complement

Let A be an NFA accepting $L(A)$. Let $A_D = (Q_D, \Sigma, \delta_D, q_{0D}, F_D)$ be a DFA accepting the same language, i.e., $L(A_1) = L(A_D)$. The automaton $\bar{A} = (Q_D, \Sigma, \delta_D, q_{0D}, Q \setminus F_D)$ accepts the complement, i.e., $(L(\bar{A}) = \Sigma^* \setminus L(A_D))$.

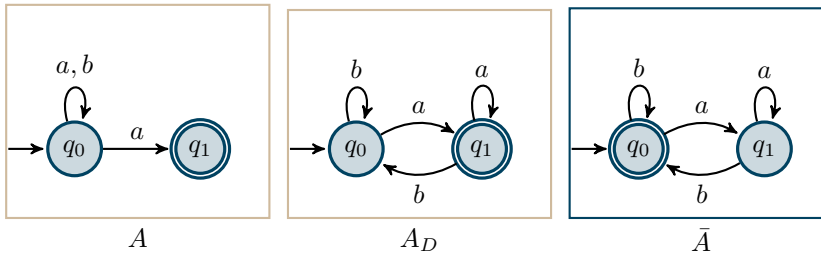


Figure 1.4: Closure under Complement

Closure under Intersection

From set theory

$$L(A_2) = \overline{\overline{L(A_1)} \cup \overline{L(A_2)}}.$$

Therefore, as we already established constructions for complement and union, this equation yields a construction for intersection.

However, this construction requires constructing three complements, each complementation requiring a subset construction to make the automata deterministic with each yielding a possibly exponential blow up of the state space.

A more efficient approach is shown in Figure 1.5. An automaton is constructed that simulates two automata in parallel. To this end, each state has two components. The first component is the state of the first automaton, the second component is the state of the second

Closure under Complement (Abschluss unter Komplement)



<https://youtu.be/0wHtNEZ6PBI>
(<https://youtu.be/5JPKSYk98s0>)

Closure under Complement – Formal Definition (Abschluss unter Komplement - Formale Definition)



<https://youtu.be/K7ap1YCxW3c>
(https://youtu.be/8IbKc_Qlj0c)

Closure under Intersection (Abschluss unter Schnitt)



<https://youtu.be/hq6Y3gCyufU>
(<https://youtu.be/P4Iqr03-3X8>)

Closure under Intersection – Formal Definition (Abschluss unter Schnitt - formale Definition)



<https://youtu.be/DARTXcLPIW8>
(<https://youtu.be/vtPiB09JTR0>)

automaton. The state space is the Cartesian product of the states of the two automata. If one automaton is stuck, the simulator is stuck.

Theorem 17: Closure under Intersection

Let $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ and $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ be non-deterministic finite automata. Define

$$A_{\times} = (Q_{\times}, \Sigma, \delta_{\times}, Q_{0\times}, F_{\times})$$

by

- $Q_{\times} = Q_1 \times Q_2$
- $\delta_{\times}((q_1, q_2), a) = \delta_1(q_1, a) \times \delta_2(q_2, a)$
- $Q_{0\times} = Q_{01} \times Q_{02}$
- $F_{\times} = F_1 \times F_2$.

Then $L(A_{\times}) = L(A_1) \cap L(A_2)$.

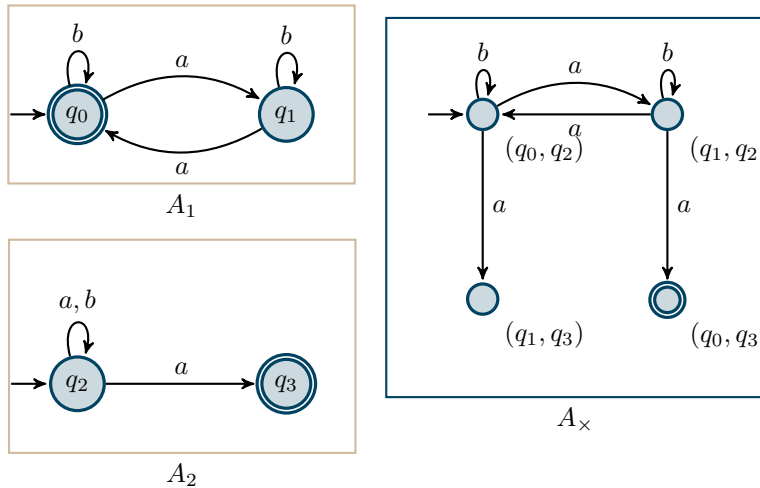


Figure 1.5: The Product Construction

For the closure properties considered so far, we only provided a construction but we did not prove, that the construction does what it is supposed to do. We considered it “obvious” that the construction was correct. For the product construction however, we will prove its correctness.

Closure under Concatenation

Concatenation is the obvious operation on strings and is easily extended to languages. In terms of finite automata this operation corresponds to sequential composition of automata. The main idea of this construction is to allow to enter the initial state of a second automaton instead of entering an accepting state in the first automaton as sketched in Figure 1.6. We use color coding to show the correspondence between the example and the formal definition in Theorem 1.4.

Correctness of the Product Construction (Beweis der Korrektheit der Produktkonstruktion)



https://youtu.be/F39QI6eNL_8
(<https://youtu.be/y9gw5KjDHDM>)

Closure under Concatenation (Abschluss unter Konkatination)



https://youtu.be/_hgtmsgFCKk
(<https://youtu.be/Td03Ke8A-34>)

Theorem 18: Closure under Concatenation

Let $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ and $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ be finite automata. Define $A_\bullet = (Q_\bullet, \Sigma, \delta_\bullet, Q_{0\bullet}, F_\bullet)$ by

- $Q_\bullet = Q_1 \cup Q_2$ (disjoint union)
- $F_\bullet = F_2$
- $Q_{0\bullet} = Q_{01}$ if $Q_{01} \cap F_1 = \emptyset$ and $Q_{0\bullet} = Q_{01} \cup Q_{02}$ otherwise .
- $\delta_\bullet(q, a) = \begin{cases} \delta_1(q, a) & \text{if } q \in Q_1 \text{ and } \delta(q, a) \cap F_1 = \emptyset \\ \delta_1(q, a) \cup Q_{02} & \text{if } q \in Q_1 \text{ and } \delta(q, a) \cap F_1 \neq \emptyset \\ \delta_2(q, a) & \text{if } q \in Q_2 \end{cases}$

Then $L(A_\bullet) = L(A_1) \cdot L(A_2)$.

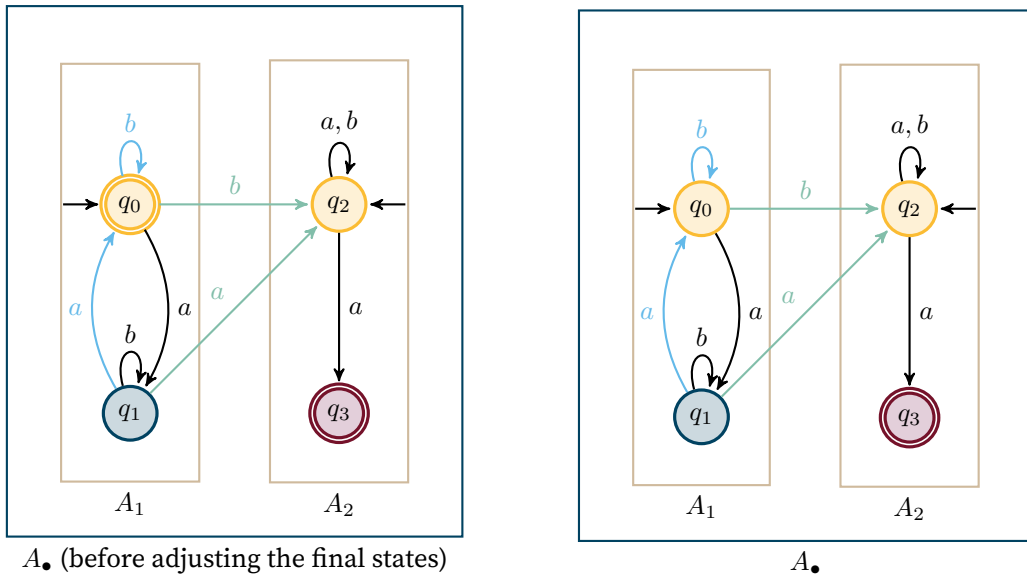


Figure 1.6: Sequential Composition of Automata

Closure under Star

The star closure collects all arbitrary concatenations of words from a language. In terms of finite automata, this operation can be realized by adding transitions that allow to choose an accepting state or going back to an initial state. This idea is illustrated in Figure 1.7.

Theorem 19: Closure under Star

Let $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ be a finite automaton. Define $A_* = (Q_*, \Sigma, \delta_*, Q_{0*}, F_*)$ by

- $Q_* = Q_1 \cup \{q_\varepsilon\}$
- $F_* = F_1 \cup \{q_\varepsilon\}$
- $Q_{0*} = Q_{01} \cup \{q_\varepsilon\}$

Closure under Star (Abschluss unter Stern)

<https://youtu.be/f-dTs5H5-uI>
(<https://youtu.be/51PDONaLjxs>)

$$\delta_*(q, a) = \begin{cases} \emptyset & \text{if } q = q_\epsilon \\ \delta(q, a) & \text{if } q \neq q_\epsilon \text{ und } \delta(q, a) \cap F_1 = \emptyset \\ \delta(q, a) \cup Q_{01} & \text{if } q \neq q_\epsilon \text{ und } \delta(q, a) \cap F_1 \neq \emptyset \end{cases}$$

Then $L(A_*) = L(A_1)^*$.

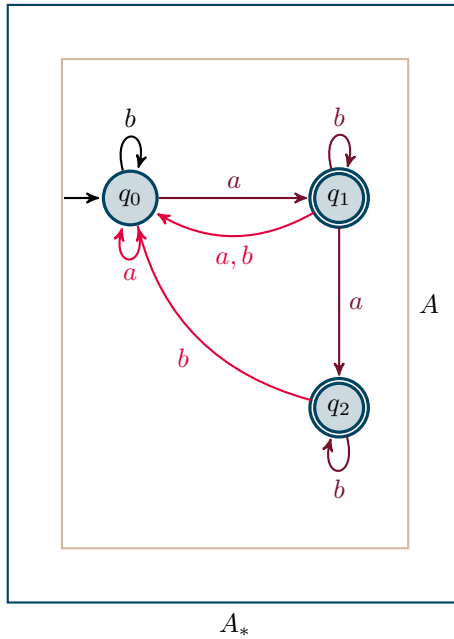


Figure 1.7: Looping Back to the Start to Accept the Star Closure

1.5 Regular Expressions

Finite automata are an operational model and can check if a word belongs to a language or not. They can be implemented easily in hardware as well as software. In practice they are usually used in the background. Specifying a language is often easier using regular expressions. A regular expression is pattern for all the words in the language. It turns out that they are equally expressive to finite automata.

Many programming languages provide build in support for regular expressions and compile them in the background into finite automata.

Regular Expressions in Java

```
import java.util.Scanner;
import java.util.regex.Pattern;

//...
public static void MACExample () {
    Scanner scanner = new Scanner(System.in);
    // regular expression for MAC addresses
    String macRegex =
    ↪ "[0-9A-Fa-f]{2}[:-]){5}([0-9A-Fa-f]{2})";
    String input = scanner.nextLine();
}
```

```
//Ex. b8:68:8f:27:6f:40
if (Pattern.matches(macRegex, input)){
    System.out.println("ok");
}
}
```

Defining Regular Expressions

The regular expressions used in practice offer a lot of character classes, operations and short cuts. This allows for a concise specification for languages in practice. We stick here to a minimal set of operation to obtain clear and simple proofs. All additional features known from regular expression in practice are just added syntactic sugar to this minimal set.

Definition 20: Regular Expressions

We provide an inductive definition of **regular expressions** E and the language $L(E)$ they define.

- *Basis*
 - $\emptyset$ is a regular expression and $L(\emptyset) = \emptyset$.
 - ε is a regular expression and $L(\varepsilon) = \{\varepsilon\}$.
 - For all $a \in \Sigma$, a is a regular expression and $L(a) = \{a\}$
- *Induction*
 - Assume E and F are regular expressions. Then,
 - $(E + F)$ is a regular expression and $L(E + F) = L(E) \cup L(F)$,
 - (EF) is a regular expression and $L(EF) = L(E) \circ L(F)$,
 - E^* is a regular expression and $L(E^*) = L(E)^*$.

To omit parentheses, we assume that concatenation has higher precedence than union.

Algebraic Laws for Regular Expressions

We consider regular expressions to be equal if they define the same language. Then many of the algebraic laws known from school maths also apply to regular expressions.

Theorem 21: Commutativity, Associativity, Distributivity

Let E, F, H be regular expressions. Then the following equalities hold.

- $E + F = F + E$,
- $E + (F + H) = (E + F) + H$,
- $E(FH) = (EF)H$,

Simple Regular Expressions in Java (Einfache Reguläre Ausdrücke in Java)



<https://youtu.be/0jdtAUbZeTQ>
(<https://youtu.be/-6GdqJ7aH8w>)

Regular Expressions i Java (Reguläre Ausdrücke in Java)



<https://youtu.be/ufMT6gb2EdQ>
(<https://youtu.be/H-NcJ6GrJqg>)

Regular Expressions (Reguläre Ausdrücke)



<https://youtu.be/9IWbFL9u-fk>
(<https://youtu.be/cYulnm8rTK4>)

Regular Expressions – Formal Definition (Reguläre Ausdrücke – Definition)



<https://youtu.be/h9h-EQTDjVQ>
(<https://youtu.be/ciKhQ6CZDK0>)

- $E(F + H) = EF + EH$ and $(E + F)H = EH + FH$.

Theorem 22: Identities

Let E be a regular expression, then

- $\emptyset + E = E$,
- $\varepsilon E = E\varepsilon = E$,
- $\emptyset E = E\emptyset = \emptyset$.

Theorem 23: Rules for Star

Let E be a regular expression, then

- $(E^*)^* = E^*$,
- $\emptyset^* = \varepsilon$,
- $\varepsilon^* = \varepsilon$,
- $E^+ \stackrel{\text{def}}{=} EE^*$,
- $E^+ + \varepsilon = E^*$.

Algebraic Laws for Regular Expressions
(Algebraische Regeln für Reguläre Ausdrücke)


<https://youtu.be/AAud20tjtNk>
<https://youtu.be/DXl13wqJHzc>

Equivalence of Regular Expressions and Finite Automata

Defining a language using a regular expression is convenient. However, it is not directly apparent how we can check if a word matches the pattern defined by a regular expression. Luckily, for every regular expression, we can construct a finite automaton accepting the same language. The resulting automaton can then be used to check if words match the expression. This is for example used in Java by the `Pattern.matches(regex, input)` method.

Conversely, we can also construct an equivalent regular expression for every finite automaton. Both formalisms define the same class of languages. This equivalence goes back to the work of S.C. Kleene.

Theorem 24: Kleene - $\Rightarrow$

For every regular expression E there is an equivalent finite automaton A such that

$$L(E) = L(A).$$

This proof simply used the known closure properties and constructions.

Proof 25: Kleene - $\Rightarrow$

Let E be a regular expression, construct an equivalent NFA A inductively:

- Basis:

– NFA for $\emptyset$:

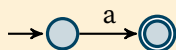


– NFA for ε :


From Regular Expression to NFA
(Vom Regulären Ausdruck zum NEA)


<https://youtu.be/soIjd20hENY>
<https://youtu.be/W3AZLHRumTE>

– NFA for a :



- Induction Hypothesis: Let E_1 and E_2 be regular expressions and A_1 and A_2 equivalent NFAs.
- Induction:
 - $E_1 + E_2$: Apply construction for the union.
 - $E_1 \cdot E_2$: Apply construction for the concatenation.
 - E_1^* : Apply construction for the star closure.

Obtaining a regular expression from a finite automaton is more difficult.

Obtaining a Regular Expression by State Elimination We can obtain a regular expression by eliminating states step-by-step until we end up with one initial and one accepting state. In the process we annotate the transitions with regular expressions.

Obtaining a Regular Expression by Solving a System of Equations Alternatively, we can derive a system of equations from a finite automaton and solve it. As these equations can be recursive, we need Arden's Lemma in the process.

1.6 Decidable Properties of Regular Languages

Finite automata are a good representation to analyze properties of regular languages.

- The word problem is to check if word belongs to a language. A DFA can simply have the word as input to check if it belongs to the language.
- The emptiness problem is to check if the language is empty. This can be checked by depth first search in the graph of the DFA.
- The equivalence problem is to check if two languages are equal. In terms of regular languages, an example would be to check if $(a+b)^*$ and $(a^*b^*)^*$ define the same language. This can also be checked using the NFA/DFA representation.

1.7 Limits of Finite Automata and the Pumping Lemma

Finite automata use the state space to store information gathered from input. This space is limited, there is no external memory. Therefore, finite automata cannot for example check if parentheses are balanced. This would require counting the number of opening parentheses and comparing with the number of closing parentheses. A finite automaton can perform this task only up to a given number determined by the construction of the automaton. If the automaton

NFA To Regular Expression by State Elimination (Idea) (Vom NEA zum regulären Ausdruck - Idee Zustandselimination)



<https://youtu.be/4tFXe74xMEM>
(<https://youtu.be/vK0-eSNsfm8>)

NFA To Regular Expression by State Elimination (Algorithm) (Vom NEA zum regulären Ausdruck - Zustandselimination (Algorithmus))



<https://youtu.be/rytI5C4Stj4>
(<https://youtu.be/kIub1Fo2JtE>)

NFA to Regular Expression by Solving Equations (Vom NEA zum regulären Ausdruck - Lösen von Gleichungen)



<https://youtu.be/rLFGAIHyxiM>
(<https://youtu.be/riIPwqlDceA>)

Decidable Properties of Regular Languages (Entscheidbare Eigenschaften regulärer Sprachen)



<https://youtu.be/0mUTLLkHiEw>
(<https://youtu.be/ghlqMe5CDnA>)

Limits of Finite Automata (Grenzen regulärer Sprachen)



<https://youtu.be/cFIASPeVQU>
(<https://youtu.be/gMUTjesEdYY>)

received more opening parentheses, the memory will be exhausted eventually.

The Pumping Lemma for Regular Languages

The Pumping Lemma is often the method of choice to show that a language is not regular. It states a property that every regular language possesses. When proving that a language is not regular, we show that the property is violated. However, this does not always work. There are non regular languages that possess the property from the Pumping Lemma.

Lemma 26: Pumping Lemma

Let L be regular. Then there is a $p \in \mathbb{N}$, such that all $z \in L$ satisfying $|z| \geq p$ can be split in $z = uvw$ where

1. $|v| \geq 1$
2. $|uv| \leq p$
3. and for all $i \in \mathbb{N}$ (incl. 0): $uv^i w \in L$.

To proof the Pumping Lemma, we consider a long word from the accepted language and observe that the automaton must have at least one state that is visited at least twice when accepting the word. Therefore, the automaton must have a loop that can then also be used arbitrarily often.

Once established, the Pumping-Lemma is used to prove that a language is not regular. This is done by considering a word with at least p characters and pumping up or down to obtain a word that is not in the language.

Proof 27: $\{a^n b^n\}$ is not regular

Assume: $L = \{a^n b^n\}$ is regular and p the constant from the Pumping Lemma. Consider $z = a^p b^p$. As $z \in L$ and $|z| \geq p$, z can be split in $z = uvw$. From the Pumping Lemma, we know that $|uv| \leq p$. Therefore, uv and consequently v can only consist of a 's. It must be of the form $v = a^k$ for a suitable k . From $|v| \geq 1$, we conclude $k \geq 1$. We pump down and obtain $uv^0 w = uw = a^{p-k} b^p \notin L$. This is a contradiction to the Pumping Lemma and L cannot be regular.

1.8 The Table-Filling-Algorithm

In deterministic finite automata every word has exactly one run. If there are two states such that there is no possible input word that would lead a non accepting state from one of them and an accepting state from the other, then there is no need to distinguish the two states. This idea can be used to minimize DFAs.

The Pumping Lemma (Das Pumping-Lemma)



<https://youtu.be/cFIASPeVQU>
(<https://youtu.be/CquKI0oofj0>)

$L = \{w \in \{a, b\}^* \mid \#_a(w) > \#_b(w)\}$ is not regular ($L = \{w \in \{a, b\}^* \mid \#_a(w) > \#_b(w)\}$ ist nicht regulär)



<https://youtu.be/28ffi04Y9cY>
(<https://youtu.be/1bcfP0Xw0Gg>)

$L = \{a^n \mid n \text{ ist prim}\}$ is not regular ($L = \{a^n \mid n \text{ ist prim}\}$ ist nicht regulär)



<https://youtu.be/9ZLBesYo7HU>
(https://youtu.be/FPNzxUg_z4w)

The Table Filling Algorithm (Der Minimierungsalgorithmus)



<https://youtu.be/nafVvug40vo>
(<https://youtu.be/2FNTDtabQz4>)

Algorithmus: The Table Filling Algorithm

Minimierungsalgorithmus

- Create a table of all unordered pairs $\{q, q'\}$, $q \neq q'$ of states.
- Mark all pairs $\{q, q'\}$ where $q \in F$ and $q' \notin F$ (or vice versa).
- While new marks are added to the table,
 - check for each unmarked pair $\{q, q'\}$ and each letter $a \in \Sigma$
 - * if $\{\delta(q, a), \delta(q', a)\}$ is already marked and mark $\{q, q'\}$ in this case.
- Merge unmarked pairs.

2

Context-Free Languages and Pushdown-Automata

Regular languages are not sufficient to describe programming languages like Java or C or a format like JSON. Seemingly simple properties like matching parentheses cannot be checked by finite automata, because they cannot count the open parentheses. Therefore, they can also not be defined by regular expressions. A more expressive formalism is needed. Today, programming languages and formats like JSON are defined using context-free grammars.

A grammar defines a set of *rules* or *productions* and allows creating words by starting with a special start symbol and successively applying these rules.

2.1 Grammars - Idea

A grammar works by replacing substrings with other substrings starting with a special start symbol. Two types of letters are distinguished: terminals and non terminals. The terminals are the symbols we want to have in the words of the language defined by the grammar. The non terminals are used in the derivation (replacement) process of creating the words in the intermediate steps, they work like variables and may not occur in the final words of the language.

The grammar in Example 29 defines the non regular language $a^n b^n$. The production $S \rightarrow aSb \mid \varepsilon$ defines, that the non terminal S can be replaced with the string aSb or the empty word. Using these rules derivations like in Example 30 can be formed.

Definition 28: Grammar and Conventions for Symbols

A **grammar** G is a structure $G = (N, \Sigma, P, S)$ where

- N is an alphabet of non terminal symbols,
- Σ is an alphabet of terminal symbols,
- P is a set productions or rules of the type $\alpha \rightarrow \beta$ and
- $S \in N$ is the start symbol.

The allowed types of rules determine the class of the grammar.

The following conventions are usually observed:

- Capital letters (A, B, C, D) are used for non terminal symbols

Welcome (Willkommen)



<https://youtu.be/N07svzrZLVQ>
(<https://youtu.be/vb4jnixZvp4>)

Grammars and Derivations (Grammatiken und Ableitungen)



https://youtu.be/D6s6o_yv0Us
(<https://youtu.be/Yu4c5oFr4w4>)

- Lower case letters (a, b, c, d) are used for terminal symbols
- Greek lower case letters (α, β, γ) are used for words that may contain terminal symbols and non terminal symbols
- Lower case letter at the end of the alphabet u, v, w, x, y, z are used for words containing only terminal symbols
- Alternatives in production rules use the symbol $|$ like in $S \rightarrow aSb \mid \varepsilon$ instead of two production rules like $S \rightarrow S \rightarrow aSb, S \rightarrow \varepsilon$.

The languages defined by a grammar is the set of all strings containing only terminal symbols that can be derived from the start symbol using the production rules.

Definition 31: Derivation

Let $G = (N, \Sigma, P, S)$ be a grammar and $\alpha_1, \alpha_2 \in (N \cup \Sigma)^*$. Then we define the derivation $\Rightarrow_G$ to be a relation on $(N \cup \Sigma)^*$ with

$$\alpha_1 \beta \alpha_2 \Rightarrow_G \alpha_1 \gamma \alpha_2 \text{ iff } \beta \rightarrow \gamma \in P$$

We write $\alpha \Rightarrow_G^* \beta$ if β can be derived from α in zero or more steps. We omit the index G if the grammar is clear from the context.

2.2 Chomsky-Hierarchy

The expressive power of grammar depends on the kind of allowed production rules. As a rule of thumb, the more restrictive we are with the type of production, the less expressive but more analyzable the grammar gets.

The Chomsky-Hierarchy classifies grammars by the structure of the allowed allowed production rules.

The most general class only requires one non terminal symbol on the left hand side of each production rule. This requirement ensures that once we have replaced all non-terminals no more production rules can be applied. The class of grammars is called Chomsky-0 and we will show in a follow up course that this type is equivalent to Turing-Machines.

Definition 32: CH-0 Grammar

A grammar $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ is type **Chomsky-0** (CH-0), iff all productions have the form

$$\alpha \rightarrow \beta$$

where $\alpha \in (N \cup \Sigma)^* N (N \cup \Sigma)^*, \beta \in (N \cup \Sigma)^*$. A language L is Chomsky-0 iff there is a Chomsky-0 grammar G generating L , i.e. $L(G) = L$.

Context sensitive grammars have rules that replace one non terminal by a string of terminals and non-terminals but consider the con-

Example 29: Grammar defining $a^n b^n$

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow aSb \mid \varepsilon\}$$

Example 30: Derivation

$$\begin{aligned} S &\Rightarrow aSb \\ &\Rightarrow aaSbb \\ &\Rightarrow aaaSbbb \\ &\Rightarrow aaaaSbbbb \\ &\Rightarrow aaaaabbbbb \end{aligned}$$

The Chomsky-Hierarchy (Die Chomsky-Hierarchy)



<https://youtu.be/xIw3KFAQKj8>
(<https://youtu.be/lTVR-lNbnfY>)

text in which the non terminal occurs. These grammars also have a corresponding machine model, the linear bound automata. But we will not further investigate this class of grammars in this course.

Definition 33: CH-1 Grammar

A grammar $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ is type Chomsky-1 (CH-1) or **context sensitive** iff all productions have the form

$$\alpha_1 A \alpha_2 \rightarrow \alpha_1 \beta \alpha_2$$

where $A \in N$, $\alpha_1, \alpha_2 \in (N \cup \Sigma)^*$, $\beta \in (N \cup \Sigma)^+$. Additionally, we allow a production rule $S \rightarrow \varepsilon$ if S does not occur on any right hand side of a production rule.

A language L is Chomsky-1 or context sensitive if there is a Chomsky-1 grammar G generating L , i.e. $L(G) = L$.

Particular relevant in practice is the class of context-free grammars and its subclasses. They are used to specify the syntax of programming languages and data formats. Productions in context-free grammars have exactly one non terminal on the left hand side in each production rule.

Definition 34: CH-2 Grammar

A grammar $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ is type Chomsky-2 or context-free, iff all production rules have the form

$$A \rightarrow \beta$$

where $A \in N$, $\beta \in (N \cup \Sigma)^*$. A language L is Chomsky-2 or context-free if there is a Chomsky-2 grammar G generating L .

Context-free grammars impose no restriction on the right hand side of production rules. By additionally requiring that the right hand side contains at most one non terminal and that this non-terminal must be the rightmost symbol on the right hand side of a rule, we obtain the class of right linear grammars. These grammars define the regular languages known from the previous part.

Definition 35: CH-3 Grammar

grammar $G = (N, \Sigma, P, S)$, $N \cap \Sigma = \emptyset$ is type Chomsky-3 or **right linear** if all production rules have the form

$$A \rightarrow aB, A \rightarrow a \text{ oder } A \rightarrow \varepsilon$$

where $A, B \in N$, $a \in \Sigma$. A language L is Chomsky-3 or regular iff there is a Chomsky-3 grammar G generating L .

A right linear production rule $A \rightarrow bB$ looks similar to an automaton transition $A \xrightarrow{b} B$. In a derivation using right linear rule, we always have exactly one non-terminal at the rightmost position of the sentential form. This is analogous to the current state of an automaton. We develop this idea in detail to show that the right linear lan-

Parse Trees and Ambiguity (Ableitungsbäume und Mehrdeutigkeit)



<https://youtu.be/ZsTITWppouk>
(<https://youtu.be/hbpcANpnuVs>)

Special Context-Free Grammars (Spezielle kontextfreie Grammatiken)



<https://youtu.be/nxqhcuuNnW0>
(https://youtu.be/th_9_Muas3o)

guages are the once accepted by finite automata.

Theorem 36: CH-3 = Regulär

A language is regular iff the language is Chomsky-3.

Proof 37: CH-3 = Regulär

only if Let L be regular. Then there is a DFA $A = (Q, \Sigma, \delta, q_0, F)$ accepting L . We define a Chomsky-3 grammar $G = (Q, \Sigma, P, q_0)$ as follows:

- For each transition $q \xrightarrow{a} q'$ we add a production rule $q \rightarrow aq'$ to P .
- For each accepting state q_f , we add a production rule $q_f \rightarrow \varepsilon$ to P .

Then $L = L(G)$.

if Let $G = (N, \Sigma, P, S)$ be a CH-3 grammar. We define an NFA $A = (N \cup \{q_f\}, \Sigma, \delta, \{S\}, F)$ accepting $L(G)$ as follows:

- For each production rule $A \rightarrow aB$, we add a transition $A \xrightarrow{a} B$.
- For each production $A \rightarrow a$, we add a production $A \xrightarrow{a} q_f$.
- Let q_f be accepting.
- For each production $A \rightarrow \varepsilon$, we let A be accepting.

Regular = Chomsky-3 (Regulär = Chomsky-3)



<https://youtu.be/VyMmpv5jURo>
(<https://youtu.be/U0Kz8KE93qc>)

2.3 Eliminating Epsilon-Productions

An epsilon production replaces a non terminal with empty word. As a consequence, derivation steps using epsilon productions are length decreasing. Such production can be problematic in parsing, so we show how these productions can be eliminated in context-free grammars.

Definition 38: ε prodction

A production $A \rightarrow \varepsilon$ is called **epsilon production**.

At first, we collect all non terminals from which we can derive the empty word in one more steps. Such non terminals are called nullable. We start with the set of all non terminals from which we can derive the empty word in one step and then extend the set until the set remains stable.

Algorithmus: Determining all nullable non terminals

1. If there is a production $A \rightarrow \varepsilon$, then A is nullable.
2. If there is a production $A \rightarrow A_1 \dots A_n$ and all $A_1, \dots, A_n$ are nullable, then A is nullable. Repeat this step until no more nullable symbols are found.

Eliminating ε Productions (Elimination von ε -Produktion)



<https://youtu.be/j9KDazY4ALU>
(<https://youtu.be/HZNwKiJIMPU>)

Algorithmus: Eliminating ϵ Productions

- For each *occurrence* of a nullable non terminal A on a right hand side, a *new* production is added with the same right hand side but with A removed. Do not add ϵ productions.
- Remove all ϵ productions.
- If S is nullable, add fresh start symbol S' and rules $S' \rightarrow S \mid \epsilon$.

Theorem 39: Testing Membership for context-free languages

Testing membership is decidable for context-free languages.

Algorithmus: Testing Membership for context-free languages

1. Eliminate all ϵ productions
2. Generate all possible derivations
 - (a) Backtrack if sentential form has been checked already
 - (b) Backtrack if sentential form is longer than w
 - (c) Return true if w is found
3. Return false if all sentential forms are checked without success

2.4 Closure Properties of context-free languages

The Javascript Object Notation (JSON) is a lightweight data-interchange format. It is defined using a context-free grammar. The complete grammar is found at json.org. We consider some of its properties as example. This example shows typical considerations when constructing grammars for a language.

- A JSON object contains a comma separated list of an arbitrary number of MEMBER. This corresponds to the Kleene star.
- Every MEMBER contains a STRING followed by a colon and an ELEMENT. This property corresponds to concatenation.
- An ELEMENT can be an OBJECT, a STRING, a NUMBER, an ARRAY (or something else). Here, we have alternatives, i.e. union.

JSON - Example

```
{
  "name": "John Smith",
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
  },
  "phoneNumbers": [
    {
      "type": "home",
```

```

    "number": "212 555-1234"
  },
  {
    "type": "office",
    "number": "646 555-4567"
  }
]
}

```

In the following section, we devise constructions yielding grammars for the union, concatenation and Kleene star.

Theorem 40: Closure under Union

The context-free languages are closed under union.

Proof 41: Closure under Union

Let $G_1 = (N_1, \Sigma, P_1, S_1)$ and $G_2 = (N_2, \Sigma, P_2, S_2)$ be context-free grammars where $N_1 \cap N_2 = \emptyset$. Let further S be a fresh non terminal, that does neither occur in N_1 nor in N_2 . Then the context-free grammar

$$G_V = (N_1 \cup N_2 \cup \{S\}, \Sigma, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}, S)$$

generates the language $L(G_1) \cup L(G_2)$.

Theorem 42: Closure under Concatenation

The context-free languages are closed under concatenation.

Proof 43: Closure under Concatenation

Let $G_1 = (N_1, \Sigma, P_1, S_1)$ and $G_2 = (N_2, \Sigma, P_2, S_2)$ be context-free grammars where $N_1 \cap N_2 = \emptyset$. Let further S be a fresh non terminal, that does neither occur in N_1 nor in N_2 . Then the context-free grammar

$$G_C = (N_1 \cup N_2 \cup \{S\}, \Sigma, P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}, S)$$

generates the language $L(G_1) \circ L(G_2)$.

Theorem 44: Closure under Kleene Star

The context-free languages are closed under Kleene Star.

Proof 45: Closure under Kleene Star

Let $G_1 = (N_1, \Sigma, P_1, S_1)$ be a context-free grammar. Let further S be a fresh non terminal not occurring in N_1 . Then the context-free grammar

$$G_S = (N_1 \cup \{S\}, \Sigma, P_1 \cup \{S \rightarrow SS_1 \mid \varepsilon\}, S)$$

generates the language $L(G_1)^*$.

Union, Concatenation, Star (*Vereinigung, Konkatenation, Stern*)



<https://youtu.be/AXy65rWPVcw>
(https://youtu.be/aXn1x_Bo1_4)

2.5 The Pumping Lemma for Context-Free languages

The pumping lemma for regular languages is the main tool for proving languages non regular. A similar lemma holds for context-free languages. If a word is in a context-free language and is long enough, at least one non terminal must occur at least twice in its derivation. This property can be used to pump to word.

Lemma 46: Pumping Lemma for context-free languages

Let L be a context-free language. There is a $p \in \mathbb{N}$ such that all $z \in L$ where $|z| \geq p$ can be divided in five parts $z = uvwxy$ satisfying

1. $|vwx| \leq p$ (the middle part has a length of at most p)
2. $|vx| \geq 1$ (v and x may not be both empty)
3. $uv^iwx^iy \in L$ for all $i \in \mathbb{N}$.

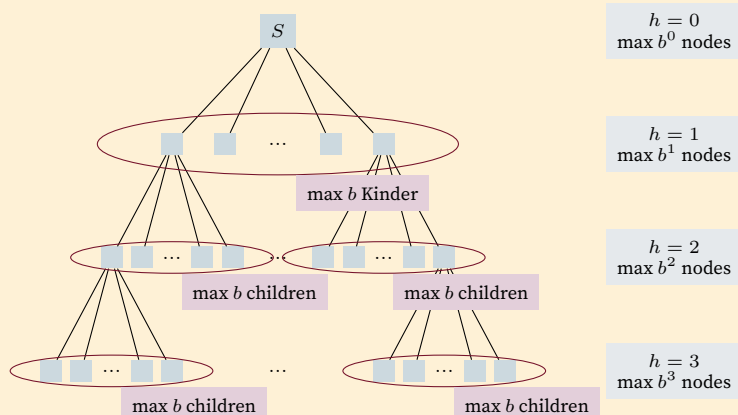
The Pumping-Lemma for context-free Languages (Das Pumping-Lemma für kontextfreie Sprachen)



<https://youtu.be/WXChgn8VsKw>
(<https://youtu.be/Y7zhIEaaTcs>)

Proof 47: Pumping Lemma for Context-Free Languages

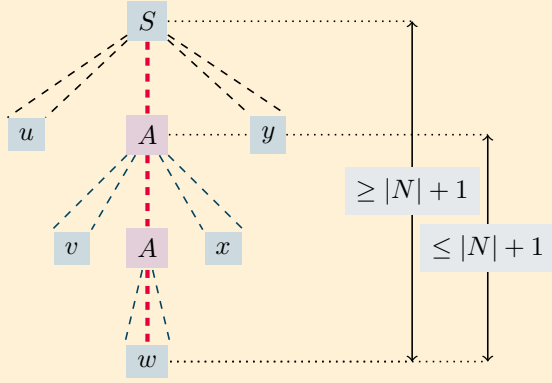
Let L be a context-free language and $G = (N, \Sigma, P, S)$ a corresponding context-free grammar and let b be the maximal number of symbols in strings occurring on the right hand sides of production rules in G . The number b is the maximal number of children a node can have in a syntax tree for a word in $L(G)$.



A syntax tree with height h can have at most b^h leaves, the leaves being the letters of the word. On the other hand, this also means that a word with a length of b^h in the language must have syntax tree with a height of at least h . We therefore define the constant

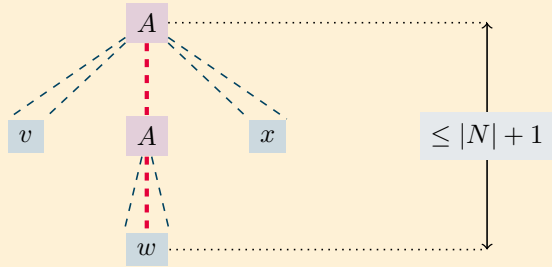
$$p = b^{|N|+1}.$$

A word $z \in L(G)$ with at least p letters ($|z| \geq p$) must have a syntax tree with a height of at least $|N| + 1$. The grammar does not need to be unambiguous, so there might be multiple syntax trees. We choose the smallest syntax tree. This smallest tree must still have a height of at least $|N| + 1$.



So there is a path in the syntax tree that contains at least $|N| + 2$ nodes – remember the root at height zero. Of these $|N| + 2$ nodes, the last node is a leaf and therefore a terminal. All other at least $|N| + 1$ nodes are non terminals. As there are only $|N|$ non terminals available, at least one non terminal – let us call it A – must occur at least twice among the last $|N| + 1$ non terminals on this path. If A occurs more than twice, we consider the last two occurrences. The syntax tree must therefore have the form depicted above. So, there is a derivation

$$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uvwxy = z.$$



In the subtree above, there can at most be $b^{|N|+1} = p$ leaves, so

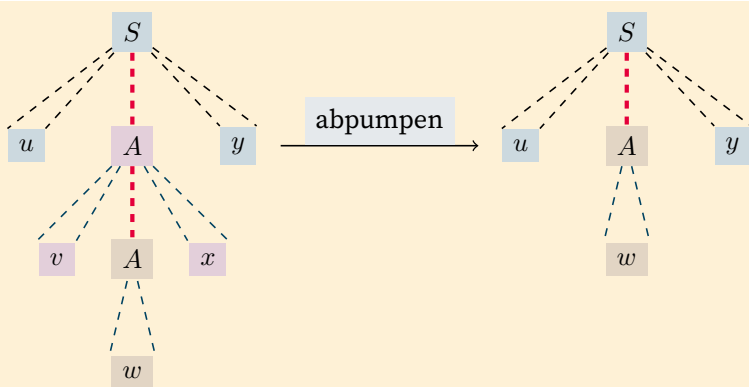
$$|vwx| \leq p.$$

As the tree for the word z is minimal, not both v and x can be the empty word because otherwise we could skip this part and obtain a smaller tree. Therefore,

$$|vx| \geq 1.$$

Removing the subtree corresponding to the derivation $A \Rightarrow^* vAx$ leads to a pumping down and we obtain the derivation

$$S \Rightarrow^* uAy \Rightarrow^* uwx.$$

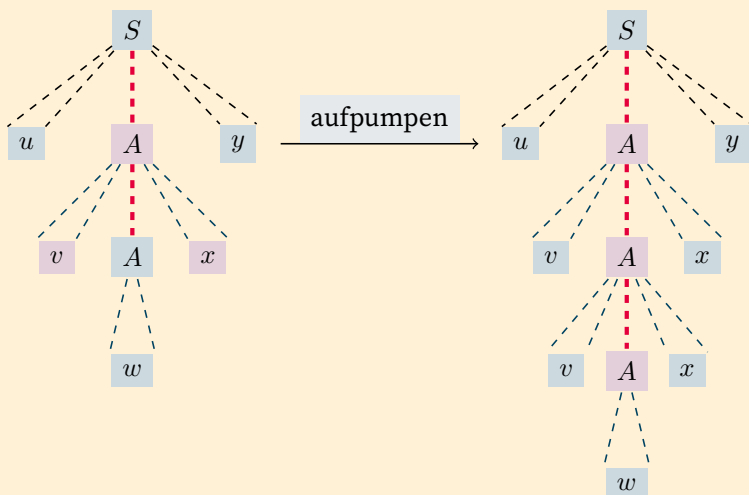


Adding the subtree once more instead, we pump um and obtain the derivation

$$S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uvvAxxxy \Rightarrow^* uvvwxxxy = uv^2wx^2y.$$

Pumping up can be done multiple times, so we get

$$uv^iwx^iy \in L(G) \text{ für alle } i \geq 0.$$



Theorem 48: $a^n b^n c^n$ is not context-free

The language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not context-free.

Proof 49: $a^n b^n c^n$ is not context-free

We assume L to be context-free. The pumping lemma guarantees the existence of a constant p . We consider the word $z = a^p b^p c^p$. As $z \in L$ and $|z| \geq p$, the word z can be divided in $z = uvwxy$. Because of the requirement $|vwx| \leq p$, we can deduce that vwx can not contain both the letter a and the letter c . One of the two letters does not occur in vwx .

Case 1: vwx does not contain an a We pump down and consider uwy . As $vx \neq \varepsilon$ some b or c have been removed but no a , so uwy has too many a .

Case 2: vwx does not contain a c We pump down and consider uwy .

$a^n b^n c^n$ is not context-free
($a^n b^n c^n$ ist nicht kontextfrei)



https://youtu.be/u83cS9rhd_Y
(https://youtu.be/zr_8A2at13g)

As $vx \neq \varepsilon$ some a or b have been removed but no c , so uwy has too many c .

In both cases uvw is not in L contrary to the pumping lemma. So L is not context-free.

2.6 Intersection and Complement

Regular languages are closed under intersection and complement, context-free languages are not.

Theorem 50: Intersection and Complement

Context-free languages are not closed under intersection and complement.

Proof 51: Intersection and Complement

The languages

$$L_1 = \{a^n b^n c^m \mid n, m \in \mathbb{N}\} \text{ and } \\ L_2 = \{a^n b^m c^m \mid n, m \in \mathbb{N}\}$$

are context-free because they are generated by the context-free grammars

$$G_1 = (\{S, A, B\}, \{a, b, c\}, \{S \rightarrow AB, A \rightarrow aAb \mid \varepsilon, B \rightarrow cB \mid \varepsilon\}, S) \\ G_2 = (\{S, A, B\}, \{a, b, c\}, \{S \rightarrow AB, A \rightarrow aA \mid \varepsilon, B \rightarrow bBc \mid \varepsilon\}, S).$$

The intersection $L_1 \cap L_2 = \{a^n b^n c^n\}$ is not context-free by application of the pumping-lemma. Therefore, the context-free languages are not closed under intersection.

The context-free languages are closed under union. If they were closed under complement they would be closed under intersection based on DeMorgans Law:

$$\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2$$

Intersection and Complement (Schnitt und Komplement)



<https://youtu.be/m2f1B49z1wI>
(<https://youtu.be/DUnYETMo9bE>)

2.7 Pushdown-Automata

Finite automata do not have memory apart from their state space. Therefore, they cannot accept languages like $a^n b^n$ or languages using balanced parentheses because they would need to keep track of the number of a or open parentheses read so far which is impossible with a finite state space.

To cope with such language, we augment our automaton model with a stack. A stack is a structure in which you can only add – push – elements on the top and access – pop – the topmost element and remove. The resulting model are pushdown automata. We only consider the non-deterministic variety. In case of pushdown automata,

Pushdown Automata (Kellerautomaten)



<https://youtu.be/3ECg6F9ssTA>
(<https://youtu.be/faYULCjpBtw>)

the non-deterministic variety is strictly more powerful than the deterministic variety.

Non deterministic pushdown automata pop the topmost element from the stack in every step. Additionally, they can either read a letter from the input or take a so-called ϵ -transition that does not consume a letter from the input. Depending on the state, the stack top and possibly an input letter, the pushdown automaton determines the successor state and can push zero or more letters onto the stack.

In our version – following Hopcroft, Ullmann – pushdown automata cannot make moves if the stack is empty. We allow different alphabets to be used for the stack and the input. Initially, we assume that a initial stack symbol is already on the stack. The transition function takes the current state, an input symbol or the ϵ and the stack top and yields the successor state and string to be placed on the stack.

Definition 52: Pushdown Automaton (PDA)

A Pushdown Automaton is a structure $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.

- Q is a finite set of **states**
- Σ is a finite **input alphabet**
- Γ is a finite **stack alphabet**
- $\delta : (Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma) \rightarrow \mathcal{P}(Q \times \Gamma^*)$ is the **transition function**
- $q_0 \in Q$ is the **initial state**
- Z_0 is the initial stack top
- $F \subseteq Q$ is the set of **accepting / final states**

The behavior of a pushdown automaton depends the current state, the stack and the remaining input. We combine this information in an instantaneous description.

Definition 53: Instantaneous Description (ID)

An **Instantaneous Description (ID)** for a pushdown automaton is a structure $(q, a_1 \dots a_n, A_1 \dots A_k) \subseteq Q \times \Sigma^* \times \Gamma^*$ comprising current state, remaining input and the stack.

We now formalize the intuitive idea of the moves of a PDA. We distinguish move that consume input and moves that do not consume input. An important consequence is that a PDA cannot make a move with empty stack.

Definition 54: Move

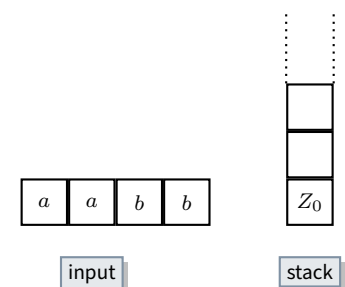
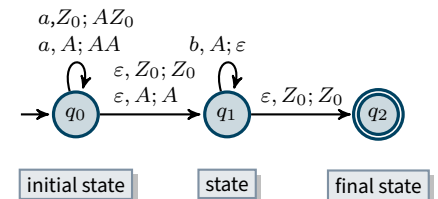
For a PDA P a move $\vdash$ is defined by

$$(q, a_1 \dots a_n, A_1 \dots A_k) \vdash (q', a_2 \dots a_n, B_1 \dots B_m A_2 \dots A_k) \\ \text{iff } \delta(q, a_1, A_1) \ni (q', B_1 \dots B_m)$$

Pushdown Automata (formal definition) (Kellerautomaten (formale Definition))



<https://youtu.be/sSVJYIuxKNA>
(<https://youtu.be/5IzTX0ZuJu4>)



and

$$(q, a_1 \dots a_n, A_1 \dots A_k) \vdash (q', a_1 \dots a_n, B_1 \dots B_m A_2 \dots A_k) \\ \text{iff } \delta(q, \epsilon, A_1) \ni (q', B_1 \dots B_m)$$

Considering accepting words, there are two equivalent approaches. We could accept if the PDA is in an accepting state after the input is consumed completely or we would accept if the PDA has an empty stack after completely consuming the input.

Definition 55: Acceptance by final state

A PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ accepts a word $w \in \Sigma^*$ by final state iff $(q_0, w, Z_0) \vdash^* (q_f, \epsilon, \alpha)$ and $q_f \in F$, i.e., if the PDA can make a sequence of moves that consumes the whole input and ends in a final state.

The language accepted by final state is

$$L(P) = \{w \in \Sigma^* \mid P \text{ accepts } w \text{ by final state}\}.$$

Definition 56: Acceptance by Empty Stack

A PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ accepts a word $w \in \Sigma^*$ by empty stack iff $(q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)$ for some arbitrary state $q \in Q$, i.e., if the PDA can make a sequence of moves that consumes the whole input and ends with an empty stack. Note that a PDA cannot make any moves once the stack is empty.

The language accepted by empty stack

$$N(P) = \{w \in \Sigma^* \mid P \text{ accepts } w \text{ by empty stack}\}.$$

The acceptance condition does not affect the expressive power of PDAs. We can convert from accepting by final state to accepting by empty stack and vice versa. For both conversions we introduce a fresh symbol as a bottom marker, that is placed on the stack before initial stack symbol is placed.

Converting from accepting by final state, we can simply empty the stack once we enter a final state. Because of the new bottom marker, an empty stack cannot arise in any state.

Converting from accepting by empty stack, we simply add a final state and add an ϵ -transition to enter this final state once the new bottom marker is found, because the bottom marker can only appear if the original PDA had an empty stack.

2.8 Pushdown Automata and Context-Free Languages

For a context-free grammar, we can easily construct a PDA accepting the same language. Constructing a grammar from a PDA is also possible but a bit more technical. The construction is omitted here but can be found in standard textbooks.

The main idea to construct a PDA from a grammar is that the PDA

A PDA for wcw^R (Kellerautomat für wcw^R)



<https://youtu.be/-DG90BMN90Y>
(<https://youtu.be/-oNk5wvmPnk>)

A PDA for Palindromes of even length (Kellerautomat für Palindrome gerader Länge)



<https://youtu.be/LnKX6UnptLI>
(https://youtu.be/rJp5rE4_qzI)

tries to derive the input word using its stack. If the stack top and next input letter agree, they can be matched. If there is a non terminal as stack top, the non terminal can be replaced by a string according to the production rules. If it is possible to derive the input using the production rule, all symbols from input and stack apart from a bottom marker will be consumed. If it is not possible, the PDA will always be stuck. Note, that this PDA is non deterministic and chooses non deterministically which production rule to apply.

Formally, the PDA has three states.

1. The first state puts the start symbol S of the grammar on top of the $\#$ bottom marker.
2. The second state matches a terminal on the stack against the next letter from the input or replaces a non terminal at the stack top by a string according to a production rule.
3. Once the bottom marker is found, the whole derived string was matched against the input and we move to the final state. If the input was consumed completely, the automaton accepts the input.

Theorem 57: PDAs and context-free grammars

For every context-free grammar G , we can construct a PDA P accepting the same language.

Proof 58: PDAs and Context-Free Languages

Let $G = (N, \Sigma, R, S)$ be a context-free grammar. Construct a PDA

$$P = (\{q_0, q_1, q_F\}, \Sigma, N \cup \Sigma \cup \{\#\}, \delta, q_0, \#, \{q_F\})$$

with three states. As the PDA simulates the derivation of the input word on the stack, the stack alphabet must comprise the terminal symbols, the non terminal symbols and the bottom marker.

We distinguish different moves:

- First move: Push start symbol on stack.
 $\delta(q_0, \varepsilon, \#) \ni (q_1, S\#)$
- Last move: Change to accepting state if bottom marker $\#$ is only symbol.
 $\delta(q_1, \varepsilon, \#) \ni (q_F, \varepsilon)$
- Match input if terminal symbol on top.
 $\delta(q_1, a, a) \ni (q_1, \varepsilon)$ for all $a \in \Sigma$
- If non terminal on top, apply production rule to replace it:
 $\delta(q_1, \varepsilon, A) \ni (q_1, B_1 \dots B_n)$ for all production rules $A \rightarrow B_1 \dots B_n$

This PDA accepts $L(G)$.

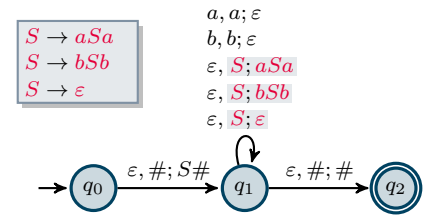


Figure 2.1: Grammar and corresponding PDA

Grammar $\rightsquigarrow$ PDA (Grammatik $\rightsquigarrow$ Kellerautomat)



<https://youtu.be/SkaTpAj5DdU>
<https://youtu.be/yXPacARHdBg>

Constructing PDAs

In some cases, it is straightforward to find a grammar but more cumbersome to find a PDA. Then the construction above provides a fail-safe way to build a corresponding PDA.

For sake of completeness, we also state the theorem that we can find a grammar for every PDA but we will omit the proof.

Theorem 59: PDAs and Context-Free Languages

For every PDA P , we can construct a grammar G accepting the same language.

3

Computability and Complexity

ving the advances and omnipresence of computers and programs in our live, one might be tempted to think that using the right software, computers can solve every problem.

However, it turns out that there are problems that are fundamentally hard that there can be no software or program to solve it. These so called undecidable problems are not esoteric but relevant in every day software development.

Once established, that a problem is at least in principle decidable by a program, the next obvious question is about the complexity of the program, .i.e. the amount of resources needed. The typical resources are time and space (memory).

3.1 Turing Machines

To tackle the question what problems are computable, we need a suitable machine model. Modern computers and programming languages would be a suitable choice. A drawback is that the devices and languages are extremely complex and therefore difficult to formalize. This would render arguments and proofs more complex.

Finite automata do not have any external memory and are therefore severely limited in their computational power, pushdown automata have a stack and can only access access the topmost element also limiting its power.

Using a finite control and an infinite tape with a movable read and write head as memory enables random access. This idea dates back to 1936 and was proposed by Alan M. Turing as a universal model of computation. It is possible to show, that our normal computers can simulate these so-called Turing-Machines and vice-versa.

Definition 60: Turing-Maschine

A Turing-Machine (TM) is a structure $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ with

- a finite set of states Q
- an input alphabet Σ
- a tape alphabet $\Gamma \supset \Sigma$

Overview (Überblick)



<https://youtu.be/1ei41hY0r5I>
(<https://youtu.be/rRSbKcUNwQ>)

Introduction to Turing-Machines (Einführung in Turing-Maschinen)



https://youtu.be/_YTT_8yVZWM
(<https://youtu.be/obwKl6y0Edg>)

- a transition function $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$ (L : left, R : right, N : neutral – no movement)
- an initial state q_0
- the blank $\sqcup \in \Gamma$ which may not occur in the input ($\sqcup \notin \Sigma$)
- a set of accepting (final) states F .

The Turing-Machine in Figure 3.1 accepts the non context-free language $a^n b^n c^n$. This input word is placed on the tape and the Turing-Machine replaces the first a by the symbol x , finding a corresponding b and c replacing both by x as well and going back to the left and starting all over until everything is marked with x . In this case the input is accepted.

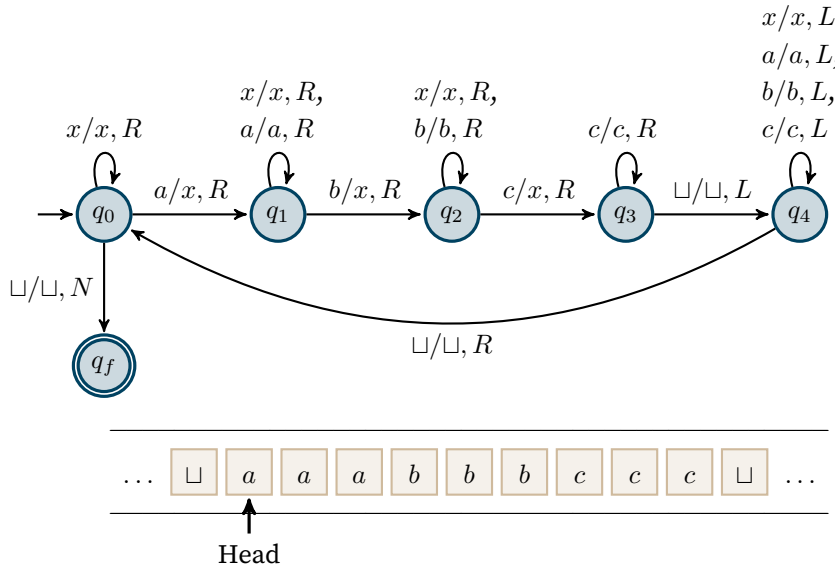


Figure 3.1: Turing-Machine accepting $a^n b^n c^n$

infinite tape
with input

To formalize the behavior of a Turing-Machine we use instantaneous descriptions similar to pushdown automata.

Definition 61: Instantaneous Description (ID)

An **instantaneous description (ID)** of a Turing-Machine defines the current state, position of the head and content of the tape. This information is defined by the string

$$uqv$$

where q is the current state, uv the tape content and the head is positioned over the first letter of v .

Turing-Machines can be used to accept languages like the other models of computation we have already encountered.

Definition 62: Accepted Language

A Turing-Machine **accepts** a word, if it can reach an accepting (or final) state, i.e.

$$(q_0 w) \rightarrow^* (\alpha q_f \beta), q_f \in F.$$

The accepted language is the set of all accepted words.

Turing-Machines can also be used to compute functions. The input for the functions is placed on the tape, then the Turing-Machine is started and if it halts eventually, the computed function value is on the tape.

Definition 63: Computed Function

A Turing-Machine computes a function if started on a suitable encoding of the input, the Turing-Machine calculates the function value and places its encoding on the tape, i.e.,

$$(q_0 w) \rightarrow^* (\alpha q_f \beta), q_f \in F$$

where w is the encoding of the input of the function and $\alpha\beta$ is the encoding of the function value for this input. Typical encodings can be unary or binary.

The Turing-Machine in Figure 3.2 multiplies two natural numbers in unary encoding, i.e., it computed

$$f : I^n \# I^m \mapsto I^{nm}.$$

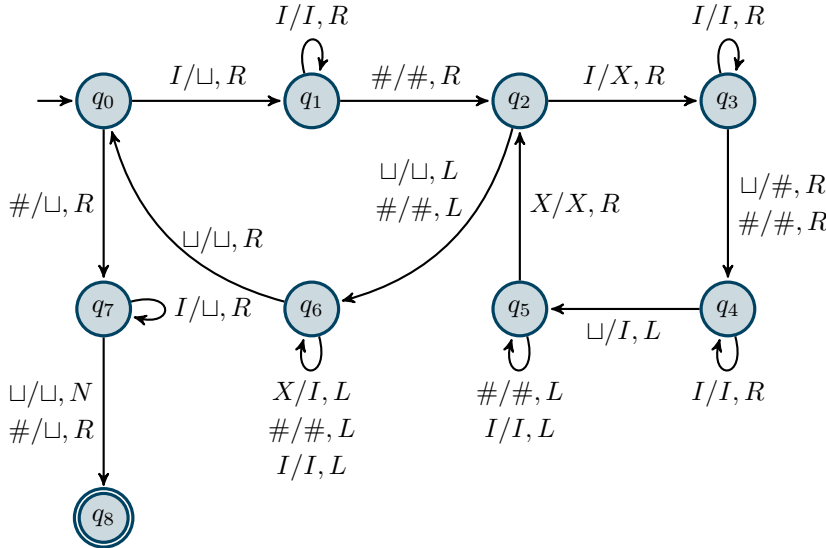


Figure 3.2: Multiplying two natural numbers using unary encoding.

Turing Machines Computing Functions (Turing-Maschinen zur Berechnung von Funktionen)



<https://youtu.be/SNnezr1kh98>
(https://youtu.be/PqaQt_ufAA0)

Multi-Tape Turing Machines

Sometimes it is convenient to consider Turing-Machines having two or more tape instead of one. This makes constructions simpler to de-

scribe and in general Turing-Machines easier to construct. The additional tapes serve as easily accesible memory. Having multiple tapes does not increase the computational power, however.

The Turing-Machine in Figure 3.3 accepts the language

$$L = \{wcw \mid w \in \{a, b\}^*\}.$$

Turing-Machines with multiple tapes expect the input on the first tape. Using this input, the TM proceeds as follows. In state q_1 , it copies the input on from first tape to the second tape going from left to right. When the TM encounters the letter c , the c is not copied to the second tape and the state changes to q_2 . In this state, the TM reserves the head of the second tape and keeps the head on the first tape on the letter c . Once the head on the second tape reaches the a blank both heads move to the right and the state changes to q_2 . This situation is depicted in the figure 3.3. In state q_2 the string on the first and second tape is matched.

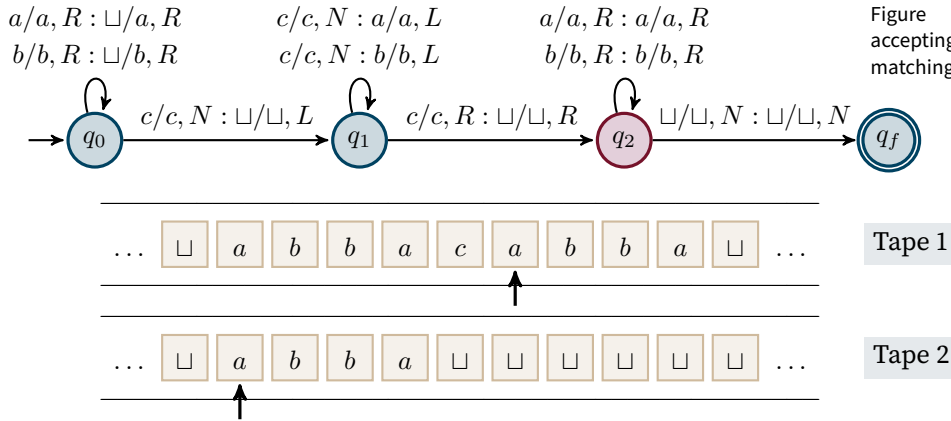


Figure 3.3: 2-Tape-Turing-Maschine accepting wcw at the beginning of the matching step.

Every multi-tape Turing-Machine can be simulated by a one-tape Turing-Machine.

The first solution is depicted in Figure 3.4. We simply align both tapes and combine the cells of the first and second tape together with head markers in one cell. Formally, we use the set of tuples as tape alphabet.

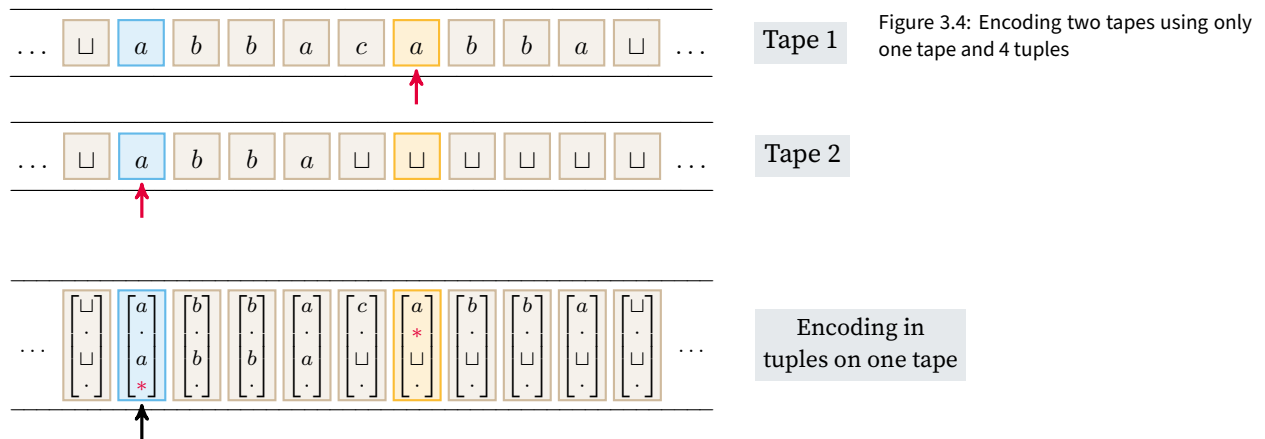
The simulating Turing-Machine has its head on the left of all tape content, moves right until it has found all head markers, computed the next state and applies the updates to heads and cells when moving back to the left.

Alternatively, we could simply put the tape contents of tape one and two simply one after the other separated by a special symbol on one tape. This idea is illustrated in Figure 3.5. As the tape is infinite there is no space issue. We also have to use markers to indicate the head positions of the simulated tapes. One problem arises if the simulated head of the first tape moves on the right beyond the simulated space on the separation marker. In this case, we have to move the whole content of the first tape to the left to make room. A similar case arises if the head of the simulated second tape moves to the left beyond its

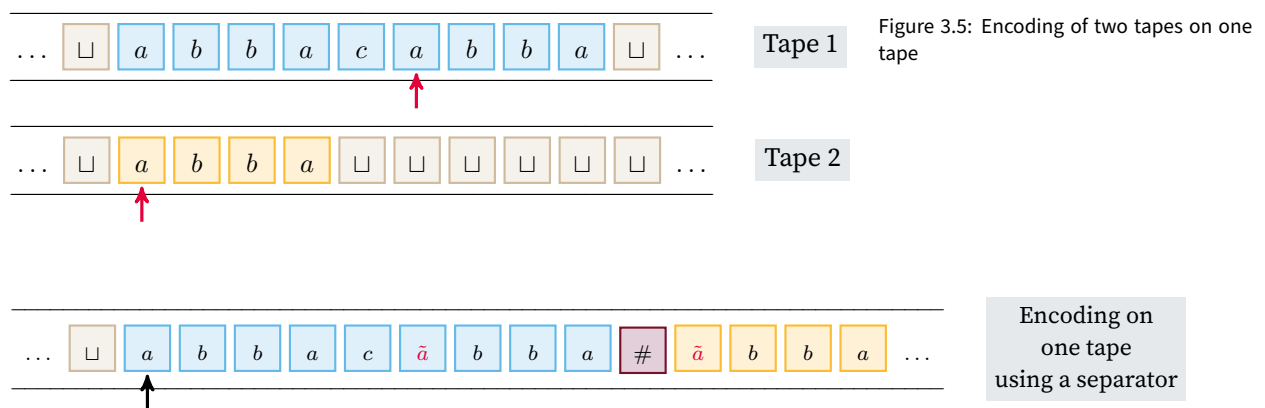
Multi Tape Turing Machines (Mehrband-Turing-Maschinen)



<https://youtu.be/5qZ9zT01Lxc>
(<https://youtu.be/m4qI5h2rsP4>)



reserved space.



Non-deterministic Turing Machines

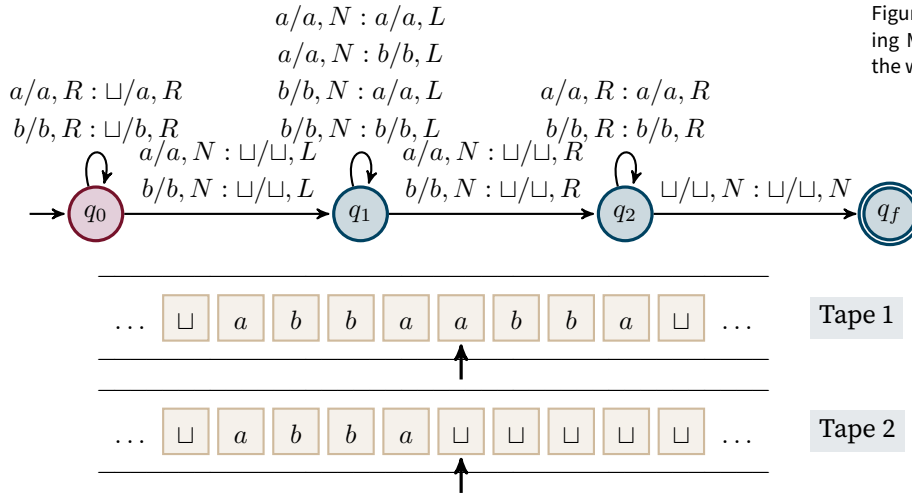
You can allow non-determinism for Turing Machines just like for finite automata and pushdown automata. Those machines are quite unhandy to implement by computers, but they often allow to come up with a concise description of an algorithmic problem. In many cases it is easier to check a given solution for validity than to find the solution. For example, it is difficult to factor a large number, but it is simple to check whether a given factorization is correct.

Non-deterministic Turing Machines first guess a solution and then check if it is correct and only accept if the check shows the guessed solution to be correct. The Turing Machine in figure 3.6 accepts the language $\{ww \mid w \in \{a,b\}^*\}$. In contrast to the language accepted in figure 3.3, we cannot use c as a marker. A non-deterministic Turing Machine has to guess the center and then needs to check if both sides have the same length. That's why this approach is known as Guess-And-Check. It is important to note that the check-step cannot be left out.

Non Deterministic Turing Machines (Nichtdeterministische Turing-Maschinen)



<https://youtu.be/ZsXvtBOVAPw>
(<https://youtu.be/6qEAd56IULY>)

Figure 3.6: Non-deterministic 2-Tape Turing Machine for ww after copying half of the word.

Deterministic Turing Machines can simulate non-deterministic Turing Machines by traversing all configurations using breadth-first search. Depth-first search cannot be used, because the simulated non-deterministic Turing Machine could have one path that doesn't terminate and another path that accepts. If the simulator used depth-first search on the infinite path, it wouldn't find the accepting path.

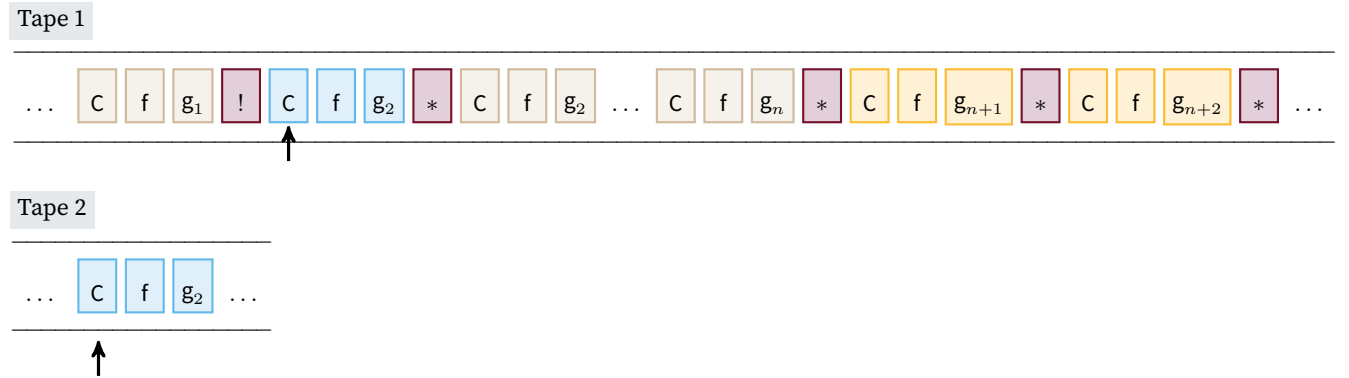


Figure 3.7: Breadth-first search in the tree of configurations with two tapes

For a breadth-first search the simulating Turing Machine depicted in figure 3.7 provides a queue on the first tape. This queue holds all configurations separated by a special symbol such as $*$ and keeps track of their heads using another special symbol such as $!$. One by one, the simulating Turing Machine copies each configuration (Cfg_2) from the queue to the working tape. Then it determines all possible successor configurations (Cfg_{n+1} and Cfg_{n+2}) and appends them to the queue. If it finds an accepting configuration of the simulated machine, the simulating machine terminates successfully. If all configurations of the simulated machine have been explored unsuccessfully, the simulating machine terminates without accepting.

3.2 Undecidable Problems

In software engineering, it's desirable to create software that does what it is supposed to do. If you call a function, you normally expect it to terminate at some point and to return control to the caller. Since we use Turing Machines as a formal model of functions within a program or as a model of the whole program, the question whether a function will terminate corresponds to the question if a Turing machine will halt. In general, this problem cannot be decided. It is undecidable. We can of course check specific functions for termination and can even prove that they terminate, but we cannot write a program that will always give us an answer to that question.

The proof is based on letting a Turing Machine run on its own code. This trick may be surprising at first, but running programs on program code – even their own – is not uncommon in practice. Program code is the input for compilers, IDEs and tools for analyzing code. For example, there's no good reason why you would not want to work on the code of the IDE Eclipse with Eclipse itself.

Encoding of Turing Machines

Turing Machines can be encoded just like programs. A Turing Machine is a tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$. We can simply write down this tuple and encode the string in binary notation – just like software code. As a result, we will have some unique encoding for each Turing Machine. If we want the encoding to be a bijection, we can assume that every binary string that does not represent a correct Turing Machine is an encoding for our favorite specific Turing Machine.

Definition 64: Encoding of Turing Machines

M_w is the Turing Machine that is encoded using $w \in \{0, 1\}^*$.

Just like the Java virtual machine or the Python interpreter can execute code, Turing Machines can simulate other Turing Machines.

The Halting Problem

A Turing Machine halts, if it cannot continue.

Definition 65: Halting of Turing Machines

A Turing Machine **halts**, if there's no transition that matches the current state and the current symbol on the tape. We assume that this is true for every accepting state.

Halting can be used as a criterion for acceptance instead of using dedicated accepting states.

Definition 66: Accepting by Halting

A Turing Machine M **accepts** a word w **by halting**, if the Turing Machine halts on input w at some point. We will denote this as $M \downarrow w$.

Turing Machines - Halting (Turing-Maschinen - Halten)



<https://youtu.be/LVx0KKpuhoc>
(<https://youtu.be/C7GHANuwkww>)

Accepting by final state and accepting by halting are equivalent approaches.

We can convert a Turing Machine that accepts by halting into a Turing Machine that accepts by final state by channeling all undefined transitions into a new accepting state.

Vice versa, we can convert a Turing Machine that accepts by final state into a Turing Machine that accepts by halting by channeling all undefined transitions of non-accepting states into a new non-final state with an infinite loop and by removing all outgoing edges from final states (if present).

The notion of *accepting* a word only requires a Turing Machine to halt in accepting state if the word is accepted, i.e. it is in the language. If the word is not accepted, the Turing Machine may either halt in some non accepting state or not halt at all. In many practical cases, it would be nice to have a definitive answer in both cases. When we observe a running Turing Machine, we do not know whether the Turing Machine will halt eventually and accepted the word or whether it runs forever. Therefore, we introduce the notion of *decidability*.

Definition 67: Decideability

A language L (or a problem) is **decideable** iff there is a Turing Machine M that on input w returns 1 as a result if $w \in L$ and 0 otherwise.

Definition 68: Special Halting Problem

The problem

Given a Turing Machine M_w , will M halt on its own code w as input is called the **special halting problem**.

Formally: The special halting problem is the language

$$K = \{w \in \{0,1\}^* \mid M_w \text{ fed with } w \text{ halts}\}$$

Theorem 69: Undecidability of the Special Halting Problem

The special halting problem is undecidable.

Proof 70: Undecidability of the Special Halting Problem

Assume K is decidable. Then there is a TM M that can decide the problem K . For this Turing Machine M running on the input x , we know

$$M \text{ running on input } x \text{ returns } \begin{cases} 1 & \text{if } M_x \text{ halts on input } x \\ 0 & \text{if } M_x \text{ does not halt on input } x \end{cases} \quad (3.1)$$

From M , we construct a Turing Machine M' that does not halt if the input Turing Machine runs on its own code. More precisely, M' performs the following steps:

1. Simulate M (result is on the tape)
2. Check the result of M

Accepting by Halting or Final State (Akzeptanz durch Halten und Endzustand)



<https://youtu.be/q0hl7nc7fy0>
(<https://youtu.be/ED2dPBTp8MI>)

Decidability (Entscheidbarkeit)



<https://youtu.be/AVJKfXA01TA>
(https://youtu.be/PKz_FGxzD7k)

Undecidability of a halting problem (Das Spezielle Halteproblem ist unentscheidbar)



<https://youtu.be/pZbUFbcGjtg>
(https://youtu.be/mUonpuXji_s)

(a) If tape = 1 $\Rightarrow$ start infinite loop

(b) If tape = 0 $\Rightarrow$ halt

The problem occurs if M' is run on its own code. Let w' be the code of the Turing Machine M' . We now observe what happens if M' runs on w' .

After constructing M' (2 (a) and 2 (b)) we have:

M' halts on input $w' \iff M$ on input w' returns 0

Due to the properties of M according to (3.1) for $x = w'$ we have:

$\iff M_{w'}$ does not halt on input w'

w' is the encoding of M' , so $M' = M_{w'}$ and thus

$\iff M'$ does not halt on input w'

This contradiction is a result of the assumption about the existence of M . Therefore, we deduce that the assumption must be false and the special halting problem is undecidable.

More Undecidable Problems

Definition 71: Halting on the Empty Tape

The problem

Given a Turing Machine M , does M halt on the empty tape is called the **halting on empty tape problem**.

More formally, the problem is the language

$$H_0 = \{w \in \{0, 1\}^* \mid M_w \text{ does halt on the empty tape as input}\}$$

Theorem 72: Halting on the Empty Tape

The halting on empty tape problem is undecidable.

Proof 73: Halting on the Empty Tape

The proof proceeds as follows: At first we show that if we could decide if a Turing Machine halts on the empty tape as input, we could also decide if a Turing Machine halts on its own code as input. As we already know, that the latter is undecidable, we can conclude that it must be undecidable if a Turing Machine halts on the empty tape.

After outlining the strategy, we start the proof. We assume to have a Turing Machine deciding if a Turing Machine M' halts on the empty tape. We want to use this decider for the halting on

Reduction - A Culinary Introduction (Reduktion - Kulinarische Einführung)



<https://youtu.be/SjE0yBQUrWY>
(<https://youtu.be/814iptNr7oc>)

Reduction - Introduction (Reduktion - Einführung)



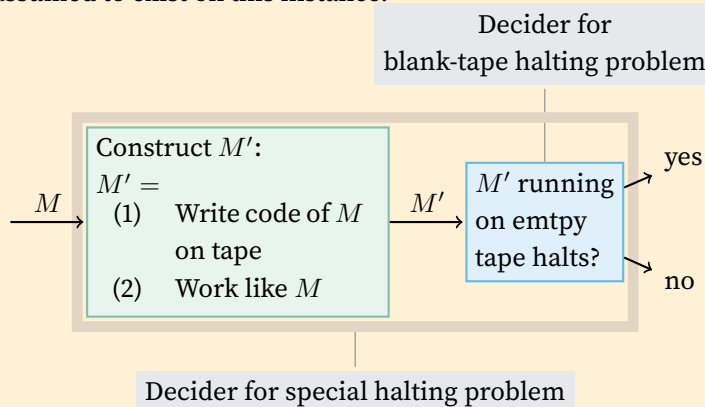
<https://youtu.be/hopVksj4-b0>
(https://youtu.be/0xf_jk893vk)

Reduction - Formal Definition (Reduktion - Formale Definition)



<https://youtu.be/ks3Jex0ht3A>
(<https://youtu.be/UufdnF5fXvI>)

empty tape problem to decide the special halting problem. We consider an instance of the special halting problem. This is a Turing Machine M for which we would like to know if M halts on its own code as input. We now have to convert M into a Turing Machine M' that halts on the empty tape if and only if M halts on its own input. This Machine M' is an instance of the halting on the empty tape problem and we can use the decider for this problem that we assumed to exist on this instance.



As shown in the figure, we construct a new Turing Machine M' from M . It first writes the code of M onto the tape and then starts M .

This new Turing Machine M' behaves on the empty tape as input just like M on the code of M as input. Therefore, M' does halt on the empty tape as input if and only if M does halt on its own code. We *reduced* the special halting problem to the halting on the empty tape problem.

If there was a decider for the blank-tape halting problem, we could decide whether a Turing Machine M would halt when running on its own code. We could do this by first constructing M' and then feed it into the decider for the halting on empty tape problem. Since it is undecidable if a Turing Machine halts on its own code as input, it is also undecidable if a Turing Machine halts on the empty tape as input.

Instead of using the schematic diagram, we can also use source code to illustrate the proof. In place of an input tape we use a variable tape. We assume that there was a function `emptytape_hp(code_l)`, that returns `True`, if the source code in parameter `code_l` halts for `tape = ''` and `False` if it doesn't halt. Using this assumption, we can write a function `special_hp(code_s)`, that checks whether `code_s` halts, if the variable `tape` holds the value `code_s`. It simply puts `'tape = '' + code_s + '\n'` in front of the other code.

Reduction of the special halting problem to the halting on the empty tape problem

```
def emptytape_hp(code_l):
    # Decides the halting on empty tape problem,
```

Halting on the Empty Tape (Leerbahthalteproblem)



https://youtu.be/AG_CinfL5kM
(<https://youtu.be/ttbMJtNsZEQ>)

```

# so whether code_l halts, if the variable
# tape holds the value ''.
tape = ''
# magic happens...
return True # or False

def special_hp(code_s):
    # Decide the special halting problem
    # for code_s, e.g. if the source code
    # code_s halts, if the variable tape holds
    # the value code_s.
    # We use the function emptytape_hp for this.
    code_l = 'tape = \'' + code_s + '\\n' + code_s
    return emptytape_hp(code_l)

```

This `code_l` now always behaves like `code_s` independent of the content of `tape` if `tape` holds the value `code_s`. If we now know if `code_l` halts for the empty tape, then we also know if `code_s` halts when `tape` holds the value `code_s`. We already know that the latter is undecidable.

You can also formally define a reduction by a function that maps an instance of one problem to an instance of another problem.

Definition 74: Reduction – formal

Let $A \subseteq \Sigma_1^*$ and $B \subseteq \Sigma_2^*$ be languages. Then A is reducible to B $A \leq B$ if there is a total and computable function $f : \Sigma_1^* \rightarrow \Sigma_2^*$, so for all $x \in \Sigma_1^*$ we have:

$$x \in A \iff f(x) \in B$$

There are three important notes regarding this definition:

1. The function f must be computable. Otherwise, the function f could already be solving an uncomputable problem.
2. The function must be total, so we know that for all instances of problem A there is an image.
3. The problems A and B must be equivalent, an implication is not sufficient. This guarantees that both cases $x \in a$ and $x \notin A$ are being covered.

Most of the time, we will not specify a function, but a construction manual on how to reduce an instance of a known problem to a new problem.

Theorem 75: Reduction

1. If B decidable and $A \leq B$, then A is decidable.
2. If A undecidable and $A \leq B$, then B is undecidable.

Proof 76: Reduction

We want to decide $x \in A$. Since $A \leq B$, there is a computable function f where $x \in A \iff f(x) \in B$. We compute $f(x)$. We decide $f(x) \in B$. That is possible, because B is decidable. Because of $x \in A \iff f(x) \in B$ we also know the answer to the question whether $x \in A$. The second part is just a logical transformation (contraposition) of the first part.

Definition 77: General Halting Problem

The problem

Given a Turing Machine M and a word w , does M halt on input w is called the general halting problem or more formally, it is the language

$$H = \{w\#x \mid M_w \text{ running on } x \text{ halts}\}$$

Theorem 78: Undecidability of the General Halting Problem

The general halting problem is undecidable.

Proof 79: Undecidability of the General Halting Problem

The function $f : \{0, 1\}^* \rightarrow \{0, 1, \#\}^*$ where $f(w) = w\#w$ is a reduction. It reduces the special halting problem to the general halting problem. Obviously, the function is total and computable. Furthermore,

$$\begin{aligned} w \in K &\iff M_w \text{ halts on input } w \\ &\iff w\#w \in H \end{aligned}$$

Definition 80: Emptiness Problem

The problem

Given a Turing-Machine M , does M not halt for any input, i.e. is the language accepted by halting empty, is called the emptiness problem. More formally, it is the language

$$E = \{w \mid M_w \text{ does not halt for any input}\}$$

Theorem 81: Undecidability of the emptiness problem

The emptiness problem is undecidable.

Proof 82: Undecidability of the emptiness problem

The strategy for the proof is as follows: We first show that if we could decide the emptiness problem, then we could also decide whether a Turing Machine halts on the empty tape as input. The latter problem is already known to be undecidable. So, we can conclude that the emptiness problem is undecidable.

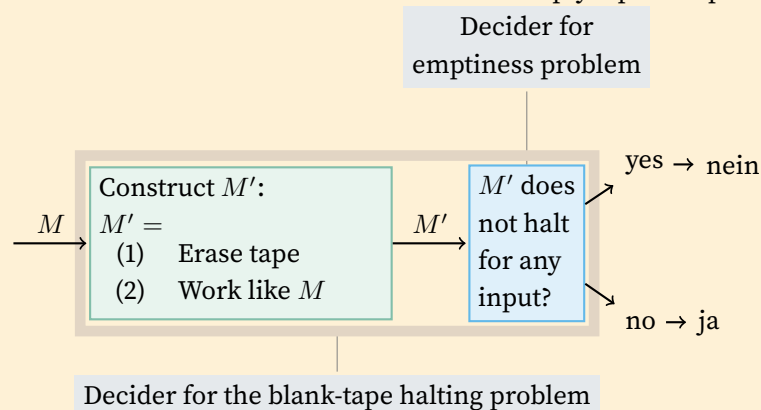
Undecidability of the general halting problem (Das allgemeine Halteproblem)

<https://youtu.be/yydMHJa8iyw>
(https://youtu.be/Msws_VAPm5A)

The Emptiness Problem (Leerheitsproblem)

<https://youtu.be/-p4uHVpjcxU>
(<https://youtu.be/PZD6yNvkd94>)

We assume that we have a decider that could determine if a Turing Machine does not halt for any input. We want to use this decider to check if a Turing Machine halts on the empty tape. Let M be an instance of the halting on the empty tape problem, i.e., a Turing Machine for which we want to determine if it halts on the empty tape. We convert this Turing Machine into a Turing Machine M' that does not halt on any input if and only if M does not halt on the empty tape. The Turing Machine M' is an instance of the emptiness problem. Knowing whether M' does not halt on any input allows us to deduce whether M halts on the empty tape as input.



As shown in the diagram, we create a new Turing Machine M' from M . This new Turing Machine will first erase the tape and then work just like M .

For all inputs, this new Turing Machine M' behaves like M on a empty tape as input. So, if M' does not halt for any input, then M does not halt on the empty tape. If M' halts for some input, then M halts on the empty tape. We realize that we have to negate the result of the decider for the emptiness problem.

The construction of M' from M is computable, so we have reduced the halting on the empty tape problem to the emptiness problem. Thus we have proven that the emptiness problem is undecidable.

This reduction can be illustrated in Python, too. If there is a function `emptiness`, then it can be used to write a function `blanktape_hp`.

Reduction of the emptiness problem to the halting on empty tape problem

```

def emptiness(code_lh):
    # Decides the emptiness problem
    # for code_lh, e.g. if the source code
    # code_lh doesn't halt for any value of variable tape.
    # magic happens

def emptytape_hp(code_l):
    # Decides if code halts on the empty input tape,
    # so if code_l halts, if the variable
    # tape holds the value ''.
    # We use the function emptiness.
  
```

```
code_lh = 'tape = ""\n' + code_l
return emptiness(code_lh)
```

When refactoring code, we would normally require that the refactored code behaves just like the original code. At the very least, we would like to have code that terminates for exactly the same inputs. In terms of Turing Machines, this problem is the equivalence problem.

Definition 83: Equivalence Problem

The problem

Given two Turing Machines M_1 and M_2 , do both Turing Machines accept the same language by halting is called the equivalence problem or more formally, it is the language

$$EQ = \{w_1 \# w_2 \mid M_{w_1} \text{ halts on input } w \text{ iff } M_{w_2} \text{ halts on input } w\}$$

Theorem 84: Undecidability of the Equivalence Problem

The equivalence problem is undecidable.

Proof 85: Undecidability of the Equivalence Problem

In order to prove that the equivalence problem is undecidable, we need a known undecidable problem that we can reduce to the equivalence problem. We will try the emptiness problem.

We start with an instance of the emptiness problem – a Turing Machine M (or an encoding thereof) that we want to know of whether it does not halt for any input.

We have to turn it into an instance of the equivalence problem. In other words: We need to find two Turing Machines M' and M'' that are equivalent according to Definition 74. It is useful to write down the equivalence explicitly.

$$\begin{aligned} &M \text{ does not halt on any input} \\ \iff &M' \text{ halts on the same words } w \text{ as } M'' \end{aligned}$$

If we have a closer look at this equivalence we observe the following: M does not halt for any input iff M accepts the same words as a Turing Machine that does not halt for any input.

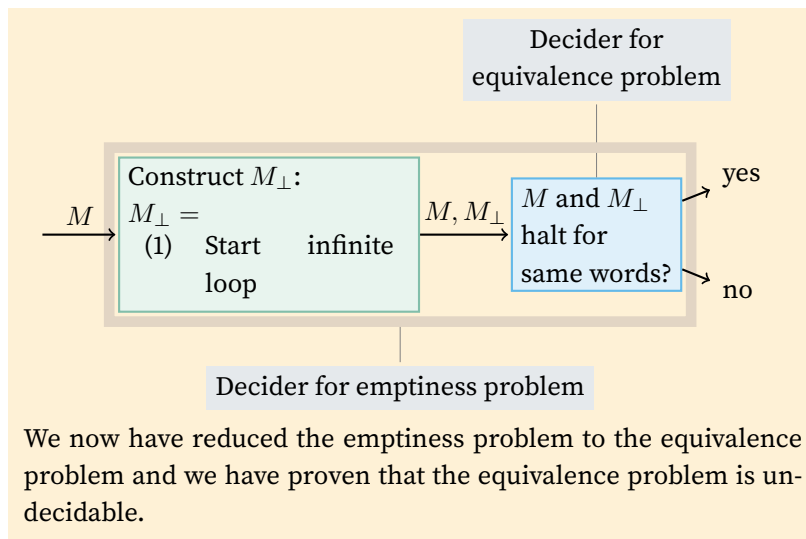
It is undecidable whether a Turing Machine does not halt for any input, but we can easily construct a Turing Machine $M_{\perp}$ with this property. In general, you cannot decide whether a program does not halt for any input. But we can easily write code that behaves exactly like that by writing an infinite loop.

So, for the reduction we can set $M' = M$ and $M'' = M_{\perp}$.

Testing Equivalence (Äquivalenzproblem)



<https://youtu.be/3TvKv9QSDEE>
(<https://youtu.be/eLURKNbQrrI>)



Rice's Theorem

In the previous section we learned that many questions related to the input/output behavior of Turing Machines are undecidable. This is not coincidence, virtually all questions about the input/output behavior of Turing Machines are undecidable.

That is Rice's Theorem. We are going to state the theorem in a function and a language acceptance version.

Theorem 86: Rice's Theorem (function version)

Let $\mathcal{R}$ be the class of all Turing-computable functions. Let $\mathcal{S}$ be a non-trivial subset, e.g. $\mathcal{S} \neq \emptyset$ and $\mathcal{S} \neq \mathcal{R}$. Then it is undecidable whether the function that's computed by a TM is within $\mathcal{S}$.

Formally:

$$C(\mathcal{S}) = \{w \mid M_w \text{ computes function in } \mathcal{S}\}$$

is undecidable.

Rice's Theorem has important consequences when it comes to programming and the correctness of programs. All the programming languages have the same expressive power as Turing Machines. So, we can translate Rice's Theorem to programming. Then the Theorem states that no matter how simple a property of a program is, it is undecidable in general if a piece of code has this property or not. There are only two exception: "The program does something" and "The program does nothing". Since programs always do something – even if they never terminate – the answer is always true in the former case and always false in the latter.

Rice's Theorem states that program verification is impossible in general. However, it is important to note two aspects. At first, Rice's Theorem does only consider input and output behavior. Questions about the structure of a program – like lines of code, etc. – are of course decidable. Secondly, Rice's Theorem only states that program

Rice's Theorem (Der Satz von Rice)



https://youtu.be/_XHqKK0Qx8
 (https://youtu.be/XPQBLTei60Y)

verification is undecidable in general. This means that there can be no general plugin for our favorite IDE that checks the correctness of our software and works in all possible cases. However, for many programs that occur in practice, verification can be possible. So, there might be plugins that work well in many cases. Developing such tools, is current research in computer science.

Proof Outline: For the proof, we reduce the halting on empty tape problem to the problem described in Rice's Theorem. Let M be a Turing Machine for which we would like to know whether it halts on the empty tape. Let furthermore Q be a Turing Machine that computes a fixed function. With these two machines, we construction their sequential composition. We build a new Turing Machine M' that first saves the input on a second tape and then starts M on the empty tape and after M halts, it starts Q on the saved input given to the Machine M' . If M halts on the empty tape, then M' just behaves like Q . If M does not halt on the empty tape, then Q is never started and M' does not halt for any input. If a Turing Machine does not halt on any input, it computes the function $\perp$ that is undefined for all inputs.

Now we just need to choose Q in a way that either Q is in S and $\perp$ is not or vice versa.

Proof 87: Rice's Theorem

Let S be an arbitrary non-trivial subset of the Turing-computable functions. Let $\perp$ be the function that is undefined everywhere. This function is computed by e.g. the Turing Machine that starts an infinite loop immediately.

For the reduction, we need an instance of the halting on the empty tape problem. So, let M be a Turing Machine for which we want to decide if M halts on the empty tape as input.

Case 1 $\perp \notin S$ Since S is not empty, at least one function q must be in S . Since all functions in S are Turing computable, there must be at least one Turing Machine Q that computes q .

We construct a Turing Machine M' that does the following:

1. Store the input x on a second tape and erase the first tape.
2. Simulate M on the now empty first tape.
3. If M halts at any time, erase the first tape and copy the original input x back to the first tape.
4. Simulate Q on the first tape.

This Turing Machine M' behaves as follows:

- If M does not halt on the empty tape, then M' also does not halt regardless of the input. In this case M' computes the function $\perp$ that is undefined for all inputs. Since $\perp \notin S$, M' computes a function that is not in S .
- If M halts on the empty tape, then M' behaves like Q , and so M' computes the function q which is in S .

Rice's Theorem - case 1

```
def rice(code_r):
    # Decide whether code_r
    # computes a function
    # in S
    # ...
    return True # or False

def emptytape_hp(code):
    # Decide the halting on
    # ↪ empty tape
    # problem with
    # the help of function
    # rice.
    #
    # Code for fixed
    # function q
    qcode = \
'tape = tape + 42'
code_r = \
'tape1 = tape\n' \
'tape = ""\n' \
'exec("'" + \
code + '"')\n' \
'tape = tape1\n' \
'exec("'" + \
qcode + '"')\n'
    return not rice(code_r)
```


Now M' computes a function in S iff M does not halt on the empty tape. So, if we could decide the problem stated in Rice's Theorem, we could also decide whether a Turing Machine halts on the empty tape. The latter is known to be undecidable

This reduction is illustrated in Python code. For the illustration, we assume that q is the function that computes $n \mapsto n + 42$.

Case 2 $\perp \in S$ This case can be treated similarly. We just need to choose q differently. Since S is the set of all Turing computable functions, there must be at least one Turing computable function q , that is **not** in S . Since q is Turing computable, there must be at least one Turing Machine Q that computes q .

We construct a Turing Machine M' just like we did for case 1. This Turing Machine behaves as follows:

- If M does not halt on the empty tape, then M' also does not halt regardless of the input. In this case M computes the function $\perp$ that is undefined for all inputs. Since $\perp \notin S$, M' computes a function that is not in S .
- If M halts on the empty tape, M' behaves like Q , so M' computes the function q which is in S .

Now M' computes a function in S iff M does not halt on the empty tape. So, if we could decide the problem stated in Rice's Theorem, we could also decide whether a Turing Machine halts on the empty tape.

We also have a look at the same Theorem in a language version, because we are often concerned with the language accepted by Turing Machines instead of the functions that are computed.

Theorem 88: Rice's Theorem (language version)

Let S be a non-trivial set of Turing-acceptable languages, i.e., $S \neq \emptyset$ and S is not the set of all Turing-acceptable languages. Then it is undecidable, if the language that is accepted by a Turing Machine is in S .

Formally:

$$L(S) = \{w \mid M_w \text{ accepts language in } S\}$$

is undecidable.

Proof 89: Rice's Theorem

Let S be an arbitrary non-trivial subset of the Turing-acceptable languages. Instead of the completely undefined function $t\perp$, we use the empty language. The empty language $\emptyset$ is accepted by e.g. the Turing Machine that starts with an infinite loop immediately. For the reduction we need an instance of the halting on the empty tape problem. So let M be a Turing Machine for which we want to

Rice's Theorem (Language Version) (Der Satz von Rice (Sprachversion))



<https://youtu.be/8RStUYyjlU>
(<https://youtu.be/DdqrcXuk7C0>)

decide whether it halts on the empty tape as input.

Case 1 $\emptyset \notin S$ Since the set S is not empty, at least one language L must be in S . Since all languages in S are Turing acceptable, there must be at least one Turing Machine M_L that accepts L .

We construct a Turing Machine M' that does the following:

1. Store the input x on a second tape and erase the first tape.
2. Simulate M on the empty first tape.
3. If M halts, erase the first tape, copy x back to the first tape.
4. Simulate M_L on the first tape.

This Turing Machine M' behaves as follows:

- If M does not halt on the empty tape, M' also does not halt regardless of the input. In this case M accepts the empty language $\emptyset$. Since $\emptyset \notin S$, M' accepts a language that is not in S .
- If M halts on the empty tape, M' behaves like M_L , so it accepts L , which is in S .

Now M' accepts a language in S iff M does not halt on the empty tape. So, if we could decide the problem stated in Rice's Theorem, we could also decide whether a Turing Machine halts on the empty tape as input.

Case 2 $\perp \in S$ This case can be treated similarly.

In the language version it is important to distinguish between $\emptyset \in S$ and $\emptyset = S$. The first case is possible, because S is a set of languages and the empty language is a Turing-accepted language. In this first case, S is not empty, because at least the empty language is in the set.

Example 90: Conclusions of Rice's Theorem

We can obtain many results directly from Rice's Theorem, e.g.

- It is undecidable if two programs do the same.
- It is undecidable if a program meets its specifications.
- It is undecidable if a Turing Machine accepts a regular language.

Application of Rice's Theorem (Anwendungen des Satz von Rice)



<https://youtu.be/XFMTb1v11J0>
(<https://youtu.be/aELKxfYs1j8>)

3.3 Semi-decidable Problems

In Definition 67 we defined decidability. It is undecidable whether a Turing Machine halts on the empty tape as input. However, we could simply start a Turing Machine with an empty tape as input. If at some point the Turing Machine halts, we know for sure that this Turing Machine halts on the empty tape. If at some point in time the Machine

Semi-Decidability (Semi-Entscheidbarkeit)



<https://youtu.be/P9mLVmMH5Gg>
(<https://youtu.be/nkU0snYeh0g>)

is still running, we do not know whether we need to wait longer for the Turing Machine to terminate or whether the Turing Machine will never halt. We only have a definite result in the positive case. This problem is undecidable but it is semi-decidable.

Definition 91: Semi-Decidability / Recursive Enumerability

A language L (or a problem) is **semi-decidable** or **recursively enumerable** iff there is a Turing Machine M that on input w returns 1 if $w \in L$. If $w \notin L$ we do not require anything except that the Turing Machine may not return 1. It may return 0 or run forever.

Theorem 92: Decidability and Semi-Decidability

1. If a language L is decidable, then it is also semi-decidable.
2. If a language L and its complement $\bar{L}$ are both semi-decidable, then L is decidable.

Proof 93: Decidability and Semi-Decidability

1. This can be directly deduced from the definition.
2. Let L and $\bar{L}$ be semi-decidable and M_1 and M_2 are the Turing Machines that semi-decide these languages. We construct a Turing Machine M , that by turn simulates M_1 on the first tape and M_2 on the second tape. If M_1 performs a step on the first tape, then M_2 performs a step on the second tape, etc.

Since the input is either in L or in the complement $\bar{L}$, one of the Turing Machines must halt eventually and write a 1. If it is M_1 , then the word is in L and the simulator M can write a 1. If M_2 terminates with result 1, then the word is not in L and the simulator M can write a 0.

3.4 Time complexity

In the previous section, we discussed the border of computability, i.e. which problems can be decided and which cannot. We did not consider how long an algorithm takes for deciding the problem. For practical problems however, the resources used for deciding a problem is crucial. We address resource consumption in this section.

Definition 94: Time requirement

The running time (or time complexity) of a Turing Machine – that halts on all inputs – is the number of steps performed. In a non-deterministic case, it is the maximum number of steps of the longest computation path – regardless of being an accepting path or not.

The number of steps usually depends on the size of the input. Therefore, the running time is usually given in O -Notation.

Complexity – Introduction (Komplexitätstheorie - Einstieg)



<https://youtu.be/h6Bm6Aiw6A8>
(https://youtu.be/r6m_Nq3USrc)

Definition 95: The Big-O-Notation

Let $g : \mathbb{N} \rightarrow \mathbb{R}_0^+$ be a function. Then $O(g)$ is the class of all functions f such that there are constants c and n_0 , such

$$f(n) < c \cdot g(n) \text{ for all } n > n_0$$

This notation describes an upper bound for the running time and provides an abstraction from the specific hardware that is used. By using the constant factor in the definition, it is irrelevant whether a machine can perform twice or three times as many operations per time unit, and it does not matter what the atomic operations look like.

For some problems, the running time has a polynomial as an upper bound. This is an important class. Polynomials are functions like n, n^2, n^3 or in general $\sum_{i=0}^n a_i x^n$.

Definition 96: The class P

The class P is the class of all languages L (problems), that can be decided by a deterministic single-tape Turing Machine in polynomial time.

The class P is robust with regard to the machine model. We are using Turing Machines, that require $O(n)$ steps for accessing a memory cell. If we used a model that could access memory in constant time ($O(1)$), we would still be dealing with the same class P .

If we also allow non-deterministic Turing Machines, we obtain the class NP .

Definition 97: The class NP

The class NP is the class of all languages L (problems), that can be decided by a non-deterministic single-tape Turing Machine in polynomial time.

When discussing finite automata, we noticed that non-deterministic machines allow to create very simple descriptions of a problem. However, they cannot be implemented in hardware directly. The same is true for Turing Machines.

One approach to simulate a non-deterministic Turing Machine by a deterministic Turing Machine is to traverse all instantaneous descriptions of the non-deterministic machines. This approach however takes exponential time.

The description of a problem using non-determinism is often more compact, because it is often easier to check a given solution for correctness than to find the solution. A non-deterministic machine would guess a solution and then check it. Therefore an alternate characterization of the class NP is: NP is the class of problems for which a solution can be checked deterministically in polynomial time.

In the section about computation theory we used the term reduction to compare problems. If a problem can be reduced to a decidable problem, then the problem itself must also be decidable.

When dealing with time complexity, we have to adapt this approach. We introduce polynomial reducibility that may require polynomial

Time Complexity – P and NP (Die Klassen P und NP)

<https://youtu.be/k6eIJkA2ohw>
(<https://youtu.be/VFqakTUBMX0>)

running time.

Definition 98: Polynomial Reducability

Let A and B be languages. Then A is **polynomial time reducible** to B $A \leq_P B$ if there is a function f that is total and computable with polynomial running time such that

$$x \in A \iff f(x) \in B.$$

(Idea: A is at most as difficult as B)

If we have a polynomial time reduction of a first problem to a second problem requiring polynomial running time, then we have a polynomial algorithm of the first problem.

Theorem 99: Polynomial Reducability

Let B be decidable in polynomial time and let A be polynomial time reducible to B . Then A can be decided in polynomial time.

If we can reduce all problems of the class NP to a specific problem, then there is no problem in NP that is “harder”. Such a problem is called NP-hard.

Definition 100: NP-Hardness

A language B is **NP-hard**, if

$$L \leq_P B$$

for all languages L in NP (B is at least as difficult as every problem in NP)

If an NP-hard problem itself is in NP, then it must be among the hardest problems in NP.

Definition 101: NP-Completeness

A language B is **NP-complete**, if B is in NP and is NP-hard.

3.5 Space complexity

Definition 102: PSpace

A language (problem) is in PSpace, if and only if it can be decided by a deterministic Turing Machine with polynomial space requirements.

Definition 103: NPSpace

A language (problem) is in NPSpace, if and only if it can be decided by a non-deterministic Turing Machine with polynomial space requirements.

Polynomialtime Reducability (Polynomialzeit Reduzierbarkeit)



<https://youtu.be/c1uwPLAz458>
(https://youtu.be/_BwhuUVj-1M)

SAT is NP-complete (SAT ist NP-vollständig)



<https://youtu.be/rbUSAKJlhrM>
(<https://youtu.be/TqGxiXV58XA>)

NP complete Problems (NP-vollständige Probleme)



<https://youtu.be/x99NCdfctug>
(<https://youtu.be/gVfgzCRvXnI>)

Rucksack is NP Complete (Rucksack ist NP-vollständig)



https://youtu.be/hNQp2_0ae3I
(<https://youtu.be/0sb-iDQnk8g>)

Theorem 104: Savitch's Theorem

Let L be a language, that can be decided by a non-deterministic Turing Machine with at least linear space requirement $f(n) \geq n$, then there is a deterministic Turing Machine with space complexity in $O(f(n)^2)$, that decides the language.

Theorem 105: PSpace = NPSpace

$\text{PSpace} = \text{NPSpace}$

Theorem 106: P, NP, PSpace

$P \subseteq NP \subseteq \text{PSpace}$

Definition 107: L

A language (problem) is in L if and only if it can be decided by a deterministic Turing Machine with $\log n$ space requirements.

Definition 108: NL

A language (problem) is in L , if and only if it can be decided by a non-deterministic Turing Machine with $\log n$ space requirements.

Definition 109: Log-Space Reducibility

A Log-Space-Transducer is a Turing Machine with one read-only input tape, one write-only output tape and one log-space working tape. A language A is log-space reducible to B if there is a function f that is total and can be computed with log-space space (by a Log-Space-Transducer) where $w \in A$ iff $f(w) \in B$

Relation Space / Time (Beziehung zwischen Platz und Zeit)

<https://youtu.be/nPj4wnWQyXM>
(<https://youtu.be/sv-c2dJRlno>)

Deterministic Space and Non-deterministic Space (Platzverbrauch im deterministischen und nichtdeterministischen Fall)

<https://youtu.be/Uce7MZkGUPK>
(https://youtu.be/8_-rINLTpoY)

PSpace = NPSpace (PSpace = NPSpace)

<https://youtu.be/nAhgTRh6YJ8>
(<https://youtu.be/218yNxw6b8g>)

Sublinear Space (Sublinearer Platzverbrauch)

<https://youtu.be/BDu72Fe61-c>
(<https://youtu.be/ghS0n33KNog>)

 $a^n b^n$ is in L ($a^n b^n$ ist in L)

<https://youtu.be/WzFr7fpdQFg>
(<https://youtu.be/d5-tdqoFUYg>)

PATH is in NL (*PATH ist in NL*)

<https://youtu.be/eZow9qnPKm4>
(<https://youtu.be/HxL8XQj7r0s>)

Summary (*Zusammenfassung*)

<https://youtu.be/tfRd22bUPX0>
(https://youtu.be/jQayJlI9_No)